

Python para Economistas

Universidad Nacional de La Plata

13 de septiembre de 2022

Temario de la clase

- 1 Introducción al curso
- 2 Introducción a Python
- 3 Instalación
- 4 Introducción a Slack
- 5 Introducción a Github
- 6 Repaso y checklist

Temario de la clase

- 1 Introducción al curso
- 2 Introducción a Python
- 3 Instalación
- 4 Introducción a Slack
- 5 Introducción a Github
- 6 Repaso y checklist

Introducción al curso

¡Bienvenidos y bienvenidas al curso Python para Economistas!

En este documento vamos a presentarnos, hacer una breve introducción al curso y compartirles algunas instrucciones de instalación para que completen antes de la primera clase.

Cualquier duda acerca de este contenido pueden consultarla por Slack, en [este grupo](#).

Si nunca usaron Slack, no se preocupen, es fácil e intuitivo. Van a recibir una invitación por correo para acceder a un *workspace* de Slack. Más adelante en esta presentación hay una explicación sobre uso básico de la plataforma.

Este curso está a cargo de Rodrigo Bonazzola y Belén Michel Torino. El contenido lo armamos en conjunto y luego nos rotamos en dar las clases. Para conocer un poco más sobre nosotros pueden encontrar una breve descripción de nuestra experiencia en [el programa](#).

Julián Borgo, Cristian Bonavida y Nicolás Abbate son los asistentes del curso. Son ex-alumnos que alcanzaron un excelente manejo de python y van a ser un apoyo clave durante la cursada.

Objetivo principal: enseñar las bases de la programación en Python para ampliar las herramientas computacionales a su disposición.

Al ser un curso introductorio, no pretende ser exhaustivo en su alcance. La intención es enseñar el conocimiento y las herramientas necesarias para que puedan continuar un camino de exploración.

El contenido está orientado a economistas, buscando proveer una base sólida de contenido teórico sobre programación junto con ejemplos aplicados que faciliten la transición de otros programas como Stata.

El contenido teórico está acompañado por ejercicio aplicados en los cuales se pondrán en práctica los conceptos y herramientas explicadas. En cada caso primero veremos ejercicios ya resueltos por nosotros, y luego habrán ejercicios a completar por ustedes.

Modalidad de cursada

La cursada está organizada en 6 clases, cada una con una parte asincrónica y otra sincrónica. El contenido asincrónico precede al contenido que veremos en los encuentros sincrónicos, por lo tanto es importante que vean los videos, practiquen y hagan llegar cualquier consulta, antes del encuentro por Zoom.

- Las clases asincrónicas (videos): se pondrán a disposición la semana previa a la clase por Zoom.
- Clases sincrónicas (zoom): Viernes de 17 a 19hs (23/9, 30/9, 7/10, 14/10, 21/10, 28/10. Clase optativa:)

Esta clase 0 es una introducción al curso. Luego tenemos las 6 clases obligatorias donde trataremos los contenidos que se detallan en el temario. Al final del curso habrá una clase 7, opcional, donde se cubrirán temas de Aprendizaje Automático y Aprendizaje Profundo.

Para aprobar el curso es necesario que participen de las clase sincrónicas, habiendo antes visto los videos de las clases asincrónicas.

Trabajos prácticos: en cada clase sincrónica, se entregará un trabajo práctico que deberá ser resuelto antes de la siguiente clase sincrónica. Algunos TPs serán individuales y otros serán grupales.

Se darán 6 días extras por si, debido a alguna dificultad personal necesitan entregar algún trabajo con retraso. Estos días pueden ser distribuidos de acuerdo a la conveniencia de los alumnos a lo largo de los seis trabajos prácticos. Los días adicionales no son fraccionables. Entregar 20 minutos o 23 horas tarde descuenta un día por igual.

Evaluación II

La nota final se basará en la participación en clase y la calificación de los trabajos prácticos, dándole un peso mayor al TP6 dado que este será integrador. La nota final será ponderada según este criterio:

- TP 1: 12 %
- TP 2: 12 %
- TP 3: 12 %
- TP 4: 12 %
- TP 5: 12 %
- TP 6: 25 %
- Participación: 15 %

Para la participación en el curso se tendrá en cuenta la participación activa (y relevante) en clases sincrónicas y Slack, la respuesta a los quiz de las clase asincrónicas y proactividad en compartir material relevante (papers, blogs, etc.).

Temario de la clase

- 1 Introducción al curso
- 2 **Introducción a Python**
- 3 Instalación
- 4 Introducción a Slack
- 5 Introducción a Github
- 6 Repaso y checklist

¿Qué es Python?

Python es un lenguaje de programación. Como tal, es un *lenguaje formal*.

Los lenguajes formales se caracterizan por una interpretación literal de un conjunto de comandos siguiendo una sintaxis específica en un *script* o *programa*.

Es un lenguaje de orden alto, es decir que está diseñado para simplificar la interpretación y uso por parte de humanos, a diferencia de lenguajes de orden bajo que son más eficientes para la interpretación por parte de computadoras.

¿Por qué Python?

Python está entre los lenguajes de código abierto más usados. Es un lenguaje versátil, con aplicaciones muy variadas.

Existe una comunidad grande de personas que usan Python para minería, análisis y ciencia de datos. Esta comunidad es importante porque lleva a que el lenguaje evolucione permanentemente e incorpore nuevas funcionalidades.

La comunidad también es una inagotable fuente de ayuda para resolver desafíos y errores de programación. Quizás el ejemplo más común es la comunidad de [Stack Overflow](#)

¿Por qué Python?

En términos prácticos, Python se diferencia de Stata en que además de poder usarse para econometría, incluye herramientas para usos variados como web scraping, natural language processing, machine learning, etc.

Dicho eso, Python, al igual que otras herramientas computacionales, deben pensarse como complementos y no necesariamente sustitutos. Cada herramienta tiene sus ventajas y desventajas. Lo que buscamos es ampliar la diversidad de instrumentos a nuestra disposición.

Temario de la clase

- 1 Introducción al curso
- 2 Introducción a Python
- 3 Instalación**
- 4 Introducción a Slack
- 5 Introducción a Github
- 6 Repaso y checklist

¿Cómo instalo Python en mi computadora?

Python es el nombre tanto del **lenguaje** en el que vamos a escribir la sintaxis cómo también del **programa** que interpreta ese lenguaje de programación en comandos accionables para la computadora. Este tipo de programas se llaman intérpretes.

Instalación: Vamos a instalar Python a través de un programa que se llama **Miniconda**. El paquete de instalación de Miniconda incluye:

- el lenguaje Python;
- un conjunto mínimo de librerías de Python (cada librería tiene herramientas para usos específicos). De hecho, se llama *Miniconda* por ser una versión liviana de Anaconda con las librerías mínimas;
- un administrador de paquetes llamado **conda** (se usa para instalar librerías nuevas);

¿Cómo instalo Python en mi computadora?

Para instalar Miniconda, por favor sigan [estas instrucciones](#) según el sistema operativo (Windows, MacOS, Linux) que usen.

Es importante que, si durante el proceso de instalación se les pregunta si agregar Miniconda al PATH de su sistema operativo, lo hagan. Esto garantiza que el sistema operativo reconozca los programas que acabamos de instalar.

Cómo usar Python para seguir las clases

Sin dudas la manera de sacar mayor provecho de este curso es que pongan en práctica lo que se ve en las clases. Los TPs contribuyen en esto, pero también es importante que sigan las clases asincrónicas en sus propias computadoras, escribiendo el código que mostramos como ejemplo y poniendo a prueba las preguntas que les puedan surgir.

¿Cómo escribir códigos en lenguaje Python? Vamos a alternar entre 3 opciones:

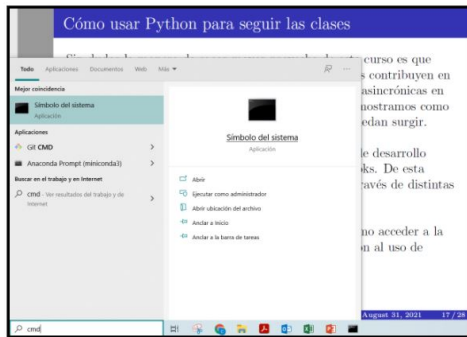
- el uso de la consola;
- Jupyter Notebooks;
- IDE (entorno de desarrollo integrado). Esencialmente un IDE es un editor de texto pero con funciones y herramientas que facilitan y mejoran la programación (i.e. llenado automático, reconocimiento de tipo de objetos, detección de errores, etc).

De esta manera van a ganar comodidad en escribir código a través de distintas interfaces.

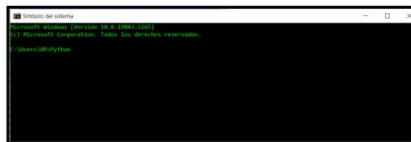
Cómo acceder a python en la consola

A continuación les detallamos los pasos a seguir.

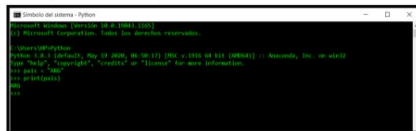
1. Escribir "cmd" en el inicio de Windows y ejecutarlo



2. Escribir "Python" en la consola y luego ENTER



3. Replicar los códigos de la clase 1



¿Cómo instalo Jupyter en mi computadora?

Jupyter es un programa que vamos a usar para escribir y correr código en las clases sincrónicas, por lo que tienen que instalarlo también. Pueden hacerlo abriendo `cmd` o la terminal de MacOS/Linux y ejecutando el siguiente comando de `conda`:

```
conda install jupyter
```

Alternativamente, pueden usar `pip install jupyter` (`pip` es el nombre de otro gestor de paquetes también incluido en Miniconda). Para chequear que la instalación fue exitosa, ejecutar `jupyter notebook`. Se debería abrir una ventana o pestaña en su navegador web, con un explorador de archivos.

Esta es la forma en la que vamos a instalar nuevos paquetes de Python en el futuro.

Temario de la clase

- 1 Introducción al curso
- 2 Introducción a Python
- 3 Instalación
- 4 **Introducción a Slack**
- 5 Introducción a Github
- 6 Repaso y checklist

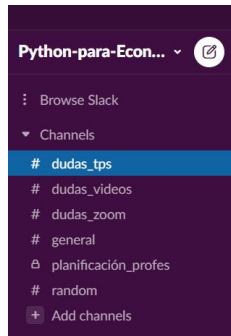
Slack es la plataforma que vamos a usar para comunicarnos. Van a recibir una invitación por correo para acceder al *workspace* del curso.

El correo va a tener un link al que tienen que acceder y seguir los pasos para crearse un usuario para la plataforma.

Slack: canales

Una vez dentro del workspace, sobre el margen izquierdo van a ver un listado de canales.

Es importante que las consultas estén dirigidas al canal correspondiente. También es importante que en primer lugar las consultas se hagan de manera pública a través del canal, no mediante mensajes privados, de manera genérica y sin compartir código de resolución de los ejercicios.

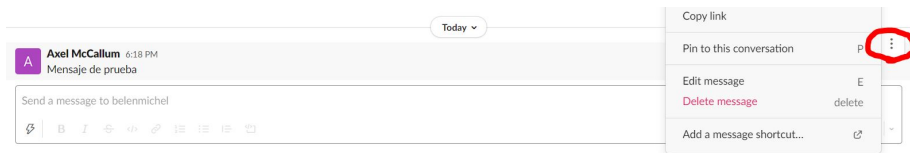


Slack: mensajes

Dentro del canal correspondiente únicamente tienen que escribir su consulta en el campo de texto ubicado en la parte baja de la ventana.



Si se equivocan, poniendo el cursor sobre el mensaje enviado, en el lado derecho aparece un menú. Haciendo click ahí se despliegan algunas opciones, incluyendo eliminar o editar el mensaje.



Slack: mensajes con código

Si llega a ser necesario mandar un mensaje con código, escriban el código usando la opción de formato que provee Slack. Por favor NO adjunten el código como imagen. Tenerlo escrito nos facilita darles una respuesta rápida editando el código original.

Si la consulta es de código, inserten el bloque de código que escribieron y no funcionó:

```
consulta = "escrita en código, no como imagen"
```



Si el mensaje es respecto a un ejercicio, enviéno mediante mensaje privado al docente a cargo de la clase. Los mensajes privados se mandan seleccionando el nombre del destinatario del listado en la parte inferior del margen izquierdo.

Temario de la clase

- 1 Introducción al curso
- 2 Introducción a Python
- 3 Instalación
- 4 Introducción a Slack
- 5 Introducción a Github**
- 6 Repaso y checklist

Github: un sistema de control de versiones

Escribir código en equipo es una habilidad muy útil para usar Python en el ámbito profesional y/o académico. Es una actividad que requiere mucha coordinación entre programadores ya que al escribir código en equipo es común trabajar con muchos archivos en simultáneo y es fácil que pequeños errores o inconsistencias pasen inadvertidos.

Para evitar estos errores involuntarios y administrar versiones de archivos modificados por múltiples usuarios se suele usar un sistema de control de versiones.

GitHub: crear cuenta e instalar

En este curso usaremos GitHub como sistema de control de versiones. Para poder usarlo, en primer lugar deben crearse una cuenta en [la plataforma](#).

Luego sigan las [las instrucciones](#) para instalar Git en su computadora.

En las próximas clases vamos a ver juntos como funciona un sistema de control de versiones y como usar GitHub para este curso.

Temario de la clase

- 1 Introducción al curso
- 2 Introducción a Python
- 3 Instalación
- 4 Introducción a Slack
- 5 Introducción a Github
- 6 Repaso y checklist

Sabemos que hay mucha información nueva. Esta ppt está para hacer una introducción al curso y guiarlos en la instalaciones necesarias. Es importante que antes del primer encuentro de la clase sincrónica hayan realizado las instalaciones necesarias y hayan completado la primera clase asincrónica que estaremos enviando en los próximos días.

Estamos para acompañarlos en este proceso. Los invitamos a que nos contacten por Slack ante cualquier duda que puedan tener.

Checklist

Les dejamos un checklist de los pasos a seguir en estos primeros días:

- Comprender el objetivo del mismo, la modalidad de cursada y modo de evaluación.
- Leer la introducción a Python y tener una noción general de por qué lo elegimos.
- Instalar Miniconda siguiendo las instrucciones provistas.
- Instalar Jupyter Notebook usando `conda` o `pip`.
- Acceder al workspace del curso en Slack.
- Crearse un usuario en GitHub
- Instalar Git en su computadora
- Mirar clase 1 asincrónica ANTES del primer encuentro el 23/9.
- Recuerden seguir los videos con la consola abierta y poniendo a prueba el contenido visto.

Python para Economistas

Universidad Nacional de La Plata
Maestría de Economía

Belén Michel Torino
Rodrigo Bonazzola

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Dato Numéricos

Enteros: o *integer* son valores numéricos enteros, positivos o negativos, de longitud ilimitada.

```
1 4
2 46821973
3 -785
```

De coma flotante: o *float* son valores numéricos con decimales, positivos o negativos. En su versión nativa guarda hasta 16 lugares decimales.

```
1 4.0214
2 4682.0008783
3 -785.1
```

Caracteres: son valores enmarcados entre ' ' o . Al estar enmarcados, los valores son interpretados como texto (y no como números u operadores aritméticos).

```
1 "A"  
2 "t"  
3 'r'  
4 "1"
```

Cadenas de Caracteres: o *string* son una sucesión de caracteres enmarcados entre ' ' o .

```
1 "Hola!"  
2 "Nos, los representantes del pueblo de la"
```

Booleano

Lógico: o *booleano* son expresiones que se evalúan a ser ciertas o no, es decir, adoptan uno de dos valores: True o False.

```
1 >>> "texto" == "texto"
2 True
3 >>> "texto" != "texto"
4 False
5 >>> 256 < 76
6 False
7 >>> 256 >= 76
8 True
9 >>> 256 >= 256
10 True
```

Tipos de datos: preguntas

Ahora veamos algunas preguntas para asentar estas definiciones. Abran una consola o un Jupyter Notebook y prueben lo siguiente para poder responder las preguntas.

¿Para Python, es igual un número escrito como entero o en formato de coma flotante? ¿Y si está escrito cómo string? Prueben lo siguiente:

```
1 >>> 17 == 17.0
2
3 >>> 524 == "524"
```

Tipos de datos: preguntas II

¿Los strings, o cadena de caracteres, son sensibles al uso de o ' '?
Prueben lo siguiente:

```
1 >>> "Argentina" == 'Argentina'
```

¿Los strings, o cadena de caracteres, son sensibles al uso de mayúsculas y minúsculas? Prueben lo siguiente:

```
1 >>> "Argentina" == "ARGENTINA"
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Operaciones básicas

Ahora veamos algunas operaciones básicas!

Sumar y Restar:

```
1 >>> 100 + 45
2 145
3 >>> 100.5 - 12.3
4 88.2
5 >>> suma_mins = 15 + 60
6 >>> suma_mins += 60 #Equivale a suma_mins = suma_mins + 60
7 ??
```

Multiplicar y Dividir:

```
1 >>> 10.1 * 3
2 30.3
3 >>> 80 / 4
4 20.0
```

Nota: división entre enteros da como resultado un float en Python 3.x.

En versiones anteriores resultaba en un entero.

Operaciones básicas II

Exponencial:

```
1 >>> 12**2
2 144
```

Cociente de una división:

```
1 >>> 10.0 // 4
2 2
```

Módulo (resto de una división):

```
1 >>> 5 % 2
2 1
3 >>> 10 % 2
4 0
5 >>> 4.5 % 4
6 0.5
```

Nota: en breve veremos un caso dónde esto es útil.

Operaciones con strings

Con *strings* también podemos hacer algunas operaciones

Concatenar (sumar):

```
1 >>> "Esto es " + "concatenar dos strings"  
2 "Esto es concatenar dos strings"  
3 >>> "a" + "bb" + "ccc"  
4 "abbccc"
```

Repetir (multiplicar):

```
1 >>> 'Muy bueno!'*2  
2 'Muy bueno!Muy bueno!'
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos**
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Operadores lógicos

Operadores lógicos: son *and*, *or* y *not* y dan como resultados un valor booleano (True o False).

Van a ser particularmente útiles para las sentencias condicionales.

```
1 >>> 4 < 7 and 4 > 1
2 True
3 >>> 4 == 1 or 4 > 1
4 True
5 >>> 4 == 1 or 4 > 9
6 False
7 >>> not (4 == 1)
8 True
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Variables

Variable: Son objetos con un *identificador* (nombre) y un contenido asociados. En Python son fáciles de definir, se hace así:

```
1 >>> mivariable = "a"
2 >>> print(mivariable)
3 "a"
4 >>> a = 100
5 >>> b = 17
6 >>> print(a + b)
7 117
```

Es una buena práctica que los nombres de las variables tengan un sentido asociado a su contenido.

```
1 >>> nombre_universidad = "Universidad de San Andres"
2 >>> print(nombre_universidad)
3 "Universidad de San Andres"
```

Variables: restricciones

Hay algunas consideraciones a tener en cuenta respecto a los nombres que se pueden utilizar al definir una variable:

- Es sensible al uso de mayúsculas/minúsculas (no son intercambiables).
- Solo puede contener valores alfanúmericos o " _"
- No puede comenzar con un caracter numérico
- No usar palabras reservadas (ej. True o False)

1

```
>>> help("keywords")
```

Algunas buenas prácticas son:

- siempre minusculas a menos que sea una constante (global en stata)
- se usa snake_case y no CamelCase
- no usar caracteres latinos (ej: "ó", "ñ", "ü", etc.).
- explicativa del valor que guarda

Operaciones y variables: preguntas

Vimos que el operador `-` se puede usar para valores numéricos y para strings. ¿Se puede usar para una combinación de ambos?

```
1 >>> 12 + "34"
```

Al nombrar dos variables de manera similar pero modificando mayúsculas/minúsculas, ¿estamos creando dos variables o pisando el valor de la primera con el valor de la segunda? *Prueben creando ambas variables y luego imprimiendo sus valores.*

```
1 >>> resultado = "a"
```

```
2 >>> RESULTADO = "b"
```

¿Alguno de estos nombres de variable es válido?

```
1 >>> 2variable = "a"
```

```
2 >>> mi-variable = "a"
```

```
3 >>> mi__var_iab_ = "a"
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

¿Que son las colecciones?

Una colección en python es un container que permite agrupar objetos y darles un nombre.

Ej. Lista de código de paises:

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
```

Hoy vamos a ver los cuatro tipos de collecciones nativas de python: listas, diccionarios, sets y tuplas.

¿Porqué tantos tipos de colecciones?

- Ordenadas vs. sin orden
- Valores repetidos vs. valores unicos
- Mutables vs. inmutables

Es una secuencia **ordenada**, **mutable** que admite **valores repetidos** y **distintos tipos** de elementos

- ¿Cómo definir una lista? con []

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
2 >>> pob_m = [44939, 11513, 211050, 18952, 7045.5, 3462]
```

- ¿saber el largo de una lista? len()

```
1 >>> len(paises)
2 6
```

- ¿Cómo acceder a un elemento? indexando (desde 0 a N o -1).

```
1 >>> paises[0]
2 'ARG'
3 >>> pob_m[-1]
4 3462
```

- ¿Cómo acceder a un sub-conjunto de elementos?

```
1 >>> paises[:]  
2 ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']  
3 >>> paises[4:]  
4 ['PRY', 'URY']  
5 >>> paises[:3] #hasta la pocision 3 sin incluir  
6 ['ARG', 'BOL', 'BRA']  
7 >>> paises[2:-2] #equivalente a paises[2:4]  
8 ['BRA', 'CHL']
```

- ¿Cómo modificar un elemento?

```
1 >>> pob_m[-2] = 7045  
2 >>> print(pob_m)  
3 [44939, 11513, 211050, 18952, 7045, 3462]
```

Listas: métodos básicos

- Agregar objetos: append, insert.

```
1 >>> paises.append('PER')
2 >>> pob_m.insert(6, 31989)
3 >>> print("La poblacion de ", paises[-1], pob_m[-1])
4 La poblacion de PER 31989
```

- Unir listas: extend

```
1 >>> paises2 = ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN']
2 >>> paises.extend(paises2)
3 >>> print(paises)
4 ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY', 'PER', 'COL',
   'ECU', 'GUY', 'SUR', 'TTO', 'VEN']
```

- Eliminar objetos: remove, del, pop.

```
1 >>> paises.remove('COL')
2 >>> paises.pop(-1)
3 'VEN'
4 >>> del paises[-4:]
```

Listas: copias y referencias

- ¿Cómo hacer una referencia de una lista?

```
1 >>> paises2 = ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN']
2 >>> paises_sa = paises2
3 >>> paises_sa.append('ARG')
4 >>> print(paises_sa)
5 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'ARG']
6 >>> print(paises2)
7 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'ARG']
```

- Sí no era esto lo que queríamos, saquemos ARG y hagamos una copia independiente de la lista paises2:

```
1 >>> paises2.remove('ARG')
```

- ¿Cómo copiar una lista?

```
1 >>> paises_sa = paises2.copy() # equivalente a:  
   paises_sa = paises2[:]  
2 >>> paises_sa.extend(paises)  
3 >>> print(paises2)  
4 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'PER']  
5 >>> paises_sa  
6 ['COL', 'ECU', 'GUY', 'SUR', 'TTO', 'VEN', 'ARG', 'BOL',  
   'BRA', 'CHL', 'PRY', 'URY', 'PER']
```


- ¿Cuál es el resultado de sumar la lista de países con la lista de población?

```
1 >>> paises + pob_m
2 ???
```

```
1 >>> paises.append(pob_m)
2
3 >>> paises.extend(pob_m)
```

Tuplas

Es una secuencia **ordenada**, **inmutable** que admite **valores repetidos** y **distintos tipos** de elementos. Se utilizan para mantener juntos varios objetos.

- ¿Cómo definir una tupla? con ()

```
1 >>> argentina = ('ARG', 44939)
2 >>> bolivia = ('BOL', 11513)
3 >>> brasil = ('BRA', 211050)
```

- ¿Cómo modificar, agregar o eliminar un elemento? No se puede.

```
1 >>> argentina[0] = 'ARGEN'
2 Traceback (most recent call last):
3   File "<stdin>", line 1, in <module>
4 TypeError: 'tuple' object does not support item
  assignment
```

```
1 >>> argentina = ('ARG', 44939, 22947)
2 >>> bolivia = ('BOL', 11513, 9086)
3 >>> brasil = ('BRA', 211050, 15259)
```

Tuplas: índices

- ¿Cómo saber el largo de una tupla? `len( )`

```
1 >>> len(brasil)
2 3
```

- ¿Cómo acceder a un elemento? indexando (desde 0 a N o -1).

```
1 >>> bolivia[0]
2 'BOL'
3 >>> bolivia[1] #equivalente a bolivia[-2]
4 11513
```

- ¿Cómo acceder a un sub-conjunto de elementos? Igual a listas

```
1 >>> argentina[:]
2 ('ARG', 44939, 22947)
3 >>> argentina[1:]
4 (44939, 22947)
5 >>> argentina[:2] #equivalente a argentina[:-1]
6 ('ARG', 44939)
```

Conjuntos (sets)

Es una secuencia **no ordenada**, **mutable** de **distintos tipos** de elementos **no repetidos**.

- ¿Cómo definir un set? con `set([ ])`

```
1 >>> chl_limit = set(['ARG', 'BOL', 'PER'])
2 >>> arg_limit = set(paises)
3 >>> print(arg_limit)
4 {'URY', 'BOL', 'ARG', 'CHL', 'PRY', 'BRA'}
5 >>> type(arg_limit)
6 <class 'set'>
```

- ¿Cómo saber el largo de un set? `len( )`

```
1 >>> len(chl_limit)
2 3
```

Sets: no ordenados

- No se puede acceder ni modificar un elemento

```
1 >>> chl_limit[:]  
2 Traceback (most recent call last):  
3   File "<stdin>", line 1, in <module>  
4 TypeError: 'set' object is not subscriptable
```

```
1 >>> chl_limit[-2] = 'AAA'  
2 Traceback (most recent call last):  
3   File "<stdin>", line 1, in <module>  
4 TypeError: 'set' object does not support item assignment
```

Sets: métodos básicos

- Agregar objetos: `add()`

```
1 >>> arg_limit.add('ARG')
2 >>> chl_limit.add('CHL')
```

- Unir dos sets: `union()`

```
1 >>> ar_cl_limit = arg_limit.union(chl_limit)
2 >>> print(ar_cl_limit)
3 {'URY', 'BOL', 'ARG', 'CHL', 'PER', 'PRY', 'BRA'}
```

- Eliminar objetos: `remove`, `del`, `pop`.

```
1 >>> arg_limit.remove('ARG')
2 >>> chl_limit.discard('CHL')
3 >>> arg_limit.pop()
4 'URY'
5 #volvamos a agregarlo: >>> arg_limit.add('URY')
```

- ¿Cómo saber la intersección y las diferencias de dos sets?

```
1 >>> arg_limit.difference(chl_limit)
2 {'CHL', 'URY', 'BRA', 'PRY'}
3 >>> arg_limit.intersection(chl_limit)
4 {'BOL'}
```

- ¿Cómo saber si un set es subset de otro o disjunto?

```
1 >>> chl_limit.issubset(arg_limit)
2 False
3 >>> chl_limit.isdisjoint(arg_limit)
4 False
```

Sets: copias y referencias

- ¿Cómo hacer una referencia de un set?

```
1 >>> pry_limit = arg_limit
2 >>> pry_limit.remove('CHL')
3 >>> arg_limit
4 {'URY', 'BOL', 'PRY', 'BRA'}
```

```
1 >>> arg_limit.add('CHL') #primero agreguemos Chile
```

- ¿Cómo hacer una copia independiente del set?

```
1 >>> pry_limit = arg_limit.copy()
2 >>> pry_limit.remove('CHL')
3 >>> arg_limit
4 {'URY', 'BOL', 'CHL', 'PRY', 'BRA'}
```


Sets y Tuplas: pregunta

- ¿El valor de temp1 y temp2 será el mismo?

```
1 >>> pry_limit
2 {'URY', 'BOL', 'PRY', 'BRA'}
3 >>> temp1 = pry_limit.remove('URY')
4 >>> temp2 = pry_limit.pop()
5
```

- ¿Cual es un posible valor de temp2?

```
1 a) {'URY', 'BOL', 'PRY'}
2 b) {'BOL', 'PRY'}
3 c) 'URY'
4 d) 'BRA'
5
```

Los diccionarios son conjuntos de **claves:valores** (key:values). Las claves deben ser **únicas y hashables**. Los **valores se pueden modificar** pero las claves no.

- ¿Cómo definir un diccionario? con { }

```
1 >>> arg_carac = {'cod': 'ARG', 'pob': 44939, 'gdp': 'N'}
2 >>> bol_carac = {'cod': 'BOL', 'pob': 11513, 'gdp': 'N'}
3 >>> print(bol_carac)
4 {'cod': 'BOL', 'pob': 11513, 'gdp': 'N'}
```

- ¿saber el largo de un diccionario? len()

```
1 >>> len(bol_carac) #equivalente a len(bol_carac.items())
2 3
```

- ¿Cómo acceder a un elemento? por la clave

```
1 >>> bol_carac['cod']  
2 'BOL'  
3 >>> arg_carac['pob']  
4 44939
```

- ¿Cómo cambiar un valor?

```
1 >>> arg_carac['gdp'] = 1031215  
2 >>> bol_carac['gdp'] = 104609
```

- ¿Cómo crear una nueva clave:valor?

```
1 >>> arg_carac['gdp_pc'] = arg_carac['gdp']/  
2                             arg_carac['pob']*100
```

- ¿Cómo borrar una clave:valor?

```
1 >>> dic_pais.pop("gdp_pc")
```

- ¿Cómo saber las claves o valores de un diccionario?

```
1 >>> bol_carac.keys()
2 dict_keys(['cod', 'poblacion', 'gdp'])
3
4 >>> bol_carac.values()
5 dict_values(['BOL', 11513, 104609])
6
7 >>> bol_carac.items()
8 dict_items([('cod', 'BOL'),
9             ('poblacion', 11513),
10            ('gdp', 104609)])
```

item ¿Cómo saber si una clave esta en un diccionario?

```
1 >>> 'cod' in bol_carac.keys()
2 True
```

- ¿Cómo obtener un valor cuando no se si la clave existe?

```
1 >>> arg_carac["gdp_pc"]
2 22946.99481519393
3
4 >>> bol_carac["gdp_pc"]
5 Traceback (most recent call last):
6   File "<stdin>", line 1, in <module>
7 KeyError: 'gdp_pc'
```

```
1 >>> bol_carac.get("gdp_pc", 'No value')
2 'No value'
```

- ¿Cómo crear un diccionario con los datos de una lista?

```
1 >>> print(países)
2 ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
3 >>> info_países = dict.fromkeys(países, "")
4 >>> info_países
5 {'ARG': '', 'BOL': '', 'BRA': '', 'CHL': '', 'PRY': '',
  'URY': ''}
```

```
1 >>> print(pob_m)
2 [44939, 11513, 211050, 18952, 7045, 3462]
3 >>> poblacion = dict(zip(países, pob_m))
4 >>> poblacion
5 {'ARG': 44939, 'BOL': 11513, 'BRA': 211050, 'CHL':
  18952, 'PRY': 7045, 'URY': 3462}
```

- Un diccionario puede ser valor de otro diccionario

```
1 >>> bra_carac = {'cod': 'BRA', 'pob': 211050, 'gdp': 3220373}
2 >>> chl_carac = {'cod': 'CHL', 'pob': 18952, 'gdp': 476738}
3 >>> pry_carac = {'cod': 'PRY', 'pob': 7045, 'gdp': 93062}
4 >>> ury_carac = {'cod': 'URY', 'pob': 3462, 'gdp': 77732}
5
6 >>> paises_info = { 'argentina': arg_carac,
7                     'bolivia': bol_carac,
8                     'brasil': bra_carac,
9                     'chile': chl_carac,
10                    'paraguay': pry_carac,
11                    'uruguay': ury_carac}
12
13 >>> paises_info['brasil']['gdp']
14 3220373
```

Cuadro 1: Resumen de colecciones

	Listas	Tuplas	Sets	Diccionarios
Se definen con	[]	()	set()	{ }
Ordenados	Si	Si	No	No
Inmutables	No	Si	No	keys: Si, values: No
Elementos unicos	No	No	Si	keys: Si, values: No
Usos	Para colecciones simples, iterables que se modifican seguido	Para datos que no van a cambiar. Puede ser la key de un diccionario.	Para chequear pertenencia y eliminar duplicados. Para elementos unicos.	Para asociaciones lógicas de key:value Para busquedas rápidas

- ¿Que pasa si intento crear un set de un diccionario?

```
1 >>> set_arg_carac = set(arg_carac)
2 >>> print(set_arg_carac)
3
4 a) error
5 b) {'cod', 'pob', 'gdp'}
6 c) {'cod', 'ARG', 'pob', 44939, 'gdp', 1031215}
7 d) {'arg_carac'}
```

Temario de la clase

- 1 Dato numéricos, de texto y booleanos
- 2 Operaciones básicas
- 3 Operadores lógicos
- 4 Interludo: definición de variables
- 5 Colecciones: Listas, Tuplas, Sets y Dicionarios
- 6 Sentencias condicionales y ciclos

Bucles: for loop

Bucle: o *loop* es cuando usamos una sentencia para indicarle al intérprete que repita una serie de instrucciones hasta que se deje de cumplir una regla definida.

for loop: para cada X de Y, hacer Z.

```
1 >>> # armo una lista de paises
2 >>> lista_paises = ["Argentina", "Bolivia", "Chile"]
3 >>> # construyo un for loop
4 >>> for pais in lista_paises: # el ":" es necesario
5 >>>     print("Nombre: " + pais) # la sangria es necesaria
6 Nombre: Argentina
7 Nombre: Bolivia
8 Nombre: Chile
```

Bucles: for loop

for loop con enumerate

```
1 >>> for n, pais in enumerate(lista_paises):  
2 >>>     print(n, ":", pais)  
3 0 : Argentina  
4 1 : Bolivia  
5 2 : Chile
```

Bucles: for loop

for loop con enumerate

```
1 >>> for n, pais in enumerate(lista_paises):  
2 >>>     print(n, ":", pais)  
3 0 : Argentina  
4 1 : Bolivia  
5 2 : Chile
```

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']  
2 >>> pob_m = [44939, 11513, 211050, 18952, 7045, 3462]
```

Bucles: for loop

for loop con enumerate

```
1 >>> for n, pais in enumerate(lista_paises):
2 >>>     print(n, ":", pais)
3 0 : Argentina
4 1 : Bolivia
5 2 : Chile
```

```
1 >>> paises = ['ARG', 'BOL', 'BRA', 'CHL', 'PRY', 'URY']
2 >>> pob_m = [44939, 11513, 211050, 18952, 7045, 3462]
```

```
1 >>> for n, pais in enumerate(paises):
2 >>>     print(pais, ': ', pob_m[n])
3 ARG: 44939
4 BOL: 11513
5 BRA: 211050
6 CHL: 18952
7 PRY: 7045
8 URY: 3462
```

Bucles: while loop

while loop: mientras X cumpla condición Y, hacer Z.

```
1 >>> # defino cupo maximo
2 >>> cupos_libres = 3
3 >>> # ahora el while loop
4 >>> while cupos_libres > 0:
5 >>>     print('Inscripcion aceptada')
6 >>>     cupos_libres -= 1
7 >>> print('Cupo lleno') # esta linea esta fuera del loop
8 Inscripcion aceptada
9 Inscripcion aceptada
10 Inscripcion aceptada
11 Cupo lleno
```

- ¿Cómo iterar sobre la lista con las poblaciones para obtener la población total de todos esos países?

Bucles: for loop

- ¿Cómo iterar sobre la lista con las poblaciones para obtener la población total de todos esos países?

```
1 >>> pob_tot = 0
2 >>> for pob in pob_m:
3 >>>     pob_tot += pob
4 >>> print(pob_tot)
5 296961
```

Control de flujo

Usamos sentencias condicionales para construir programas (scripts) que respondan a condiciones predefinidas.

sentencia condicional: if A, X, elif B, Y, else C, Z.

```
1 >>> edad = 25
2 >>> # aca empieza la sentencia condicional
3 >>> if edad < 18: # el ":" es importante
4 >>>     print("Menor de edad") # atencion a la indentaci n
5 >>> elif edad >= 18 and edad <= 30:
6 >>>     print("Mayor de edad")
7 >>> else:
8 >>>     print("No se admiten Mayores de 30")
9 Mayor de edad
```

Sentencias: juntamos lo visto

Los tipos de sentencia se pueden combinar.

```
1 >>> listado_numeros = [10,11,12,13]
2 >>> # combinamos un for loop y un if
3 >>> for numero in listado_numeros:
4 >>>     if numero % 2 == 0: # se acuerdan del modulo?
5 >>>         print(numero, "es par")
6 >>>     else:
7 >>>         print(numero, "es impar")
8 10 es par
9 11 es impar
10 12 es par
11 13 es impar
```

Dudas?

Python para Economistas

Universidad Nacional de La Plata

Septiembre, 2022

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Funciones

Son objetos que sirven para modularizar el código, haciéndolo más reutilizable y fácil de leer, corregir y mantener.

Para crear una función comenzamos con 'def', luego un nombre para la función, luego entre () ponemos los argumentos y finalmente ':'

Funciones

Son objetos que sirven para modularizar el código, haciendolo más reutilizable y fácil de leer, corregir y mantener.

Para crear una función comenzamos con 'def', luego un nombre para la función, luego entre () ponemos los argumentos y finalmente ':'

```
1 >>> def gdp_per_capita(gdp, poblacion):  
2 >>>     gdp_pc = round(gdp/poblacion*1000)  
3 >>>     return gdp_pc
```

Funciones

Son objetos que sirven para modularizar el código, haciéndolo más reutilizable y fácil de leer, corregir y mantener.

Para crear una función comenzamos con 'def', luego un nombre para la función, luego entre () ponemos los argumentos y finalmente ':'

```
1 >>> def gdp_per_capita(gdp, poblacion):  
2 >>>     gdp_pc = round(gdp/poblacion*1000)  
3 >>>     return gdp_pc
```

Para usar una función escribimos el nombre y entre () ponemos los argumentos

```
1 >>> gdp_per_capita(77732, 3462)  
2 22453  
3 >>> gdp_per_capita(ury_carac['gdp'], ury_carac['pob'])  
4 22453
```

Los atributos de las funciones pueden tener un valor predefinido o no.

Los atributos de las funciones pueden tener un valor predefinido o no.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False):
2 >>>     if gdp_billion: #equivale a if gdp_billion==True:
3 >>>         gdp_pc = round(gdp*1000000/pob)
4 >>>     else:
5 >>>         gdp_pc = round(gdp/pob)
6 >>>     return gdp_pc
```

Los atributos de las funciones pueden tener un valor predefinido o no.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False):  
2 >>>     if gdp_billion: #equivale a if gdp_billion==True:  
3 >>>         gdp_pc = round(gdp*1000000/pob)  
4 >>>     else:  
5 >>>         gdp_pc = round(gdp/pob)  
6 >>>     return gdp_pc
```

Para usar esta función escribimos el nombre y entre () ponemos los argumentos

Los atributos de las funciones pueden tener un valor predefinido o no.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False):
2 >>>     if gdp_billion: #equivale a if gdp_billion==True:
3 >>>         gdp_pc = round(gdp*1000000/pob)
4 >>>     else:
5 >>>         gdp_pc = round(gdp/pob)
6 >>>     return gdp_pc
```

Para usar esta función escribimos el nombre y entre () ponemos los argumentos

```
1 >>> print(gdp_per_capita(445445000000, 44938712))
2 9912.27
3 >>> print(gdp_per_capita(445445, 44938712, True))
4 9912.27
```

Los valores predefinidos generan que a veces no sea suficiente con especificar los valores de los atributos solo por la posición.

Los valores predefinidos generan que a veces no sea suficiente con especificar los valores de los atributos solo por la posición.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False,
2                        gdp_millon=False):
3 >>>     if gdp_billion: #equivale a if gdp_billion==True:
4 >>>         gdp_pc = round(gdp*1000000/pob)
5 >>>     elif gdp_millon:
6 >>>         gdp_pc = round(gdp*1000/pob)
7 >>>     else:
8 >>>         gdp_pc = round(gdp/pob)
9 >>>     return gdp_pc
```


Funciones

Los valores predefinidos generan que a veces no sea suficiente con especificar los valores de los atributos solo por la posición.

```
1 >>> def gdp_per_capita(gdp, pob, gdp_billion=False,
2                       gdp_millon=False):
3 >>>     if gdp_billion: #equivale a if gdp_billion==True:
4 >>>         gdp_pc = round(gdp*1000000/pob)
5 >>>     elif gdp_millon:
6 >>>         gdp_pc = round(gdp*1000/pob)
7 >>>     else:
8 >>>         gdp_pc = round(gdp/pob)
9 >>>     return gdp_pc

1 >>> print(gdp_per_capita(445445000000, 44938712))
2 9912.27
3 >>> print(gdp_per_capita(445445, 44938712, True, False))
4 9912.27
5 >>> print(gdp_per_capita(445445, 44938712, gdp_millon=True))
6 9912.27
```

Funciones: DocString

Un `DocString` es una de cadena de caracteres (`strings`) o varias líneas, delimitado por comillas al comienzo de un módulo, clase, método o función que describe lo que hace la función.

Funciones: DocString

Un DocString es una de cadena de caracteres (**strings**) o varias líneas, delimitado por comillas al comienzo de un módulo, clase, método o función que describe lo que hace la función.

```
1 >>> def gdp_per_capita(gdp, poblacion):
2 >>>     """
3 >>>     Esta funcion calcula el GDP per Capita
4 >>>     Input:
5 >>>         gdp(int): el GDP de un pais x
6 >>>         poblacion(int): la poblacion de un pais x
7 >>>     Output:
8 >>>         gdp_pc(int): el gdp per capita del pais x
9 >>>     """
10 >>>     gdp_pc = round(gdp/poblacion*1000)
11 >>>     return gdp_pc

1 >>> help(gdp_per_capita)
```

Funciones: Pregunta

Definamos un diccionario con la información de algunos países:

```
1 >>> {'argentina': {'cod': 'ARG', 'pob': 44939, 'gdp':  
1031215}, 'bolivia': {'cod': 'BOL', 'pob': 11513, 'gdp':  
104609}, 'brasil': {'cod': 'BRA', 'pob': 211050, 'gdp':  
3220373}, 'chile': {'cod': 'CHL', 'pob': 18952, 'gdp':  
476738}, 'paraguay': {'cod': 'PRY', 'pob': 7045, 'gdp':  
93062}, 'uruguay': {'cod': 'URY', 'pob': 3462, 'gdp':  
77732}}
```

Funciones: Pregunta

Definamos un diccionario con la información de algunos países:

```
1 >>> {'argentina': {'cod': 'ARG', 'pob': 44939, 'gdp':  
1031215}, 'bolivia': {'cod': 'BOL', 'pob': 11513, 'gdp':  
104609}, 'brasil': {'cod': 'BRA', 'pob': 211050, 'gdp':  
3220373}, 'chile': {'cod': 'CHL', 'pob': 18952, 'gdp':  
476738}, 'paraguay': {'cod': 'PRY', 'pob': 7045, 'gdp':  
93062}, 'uruguay': {'cod': 'URY', 'pob': 3462, 'gdp':  
77732}}
```

1) ¿Cómo usarían la función `gdp_per_capita` para agregar el gdp per capita a cada país del diccionario `'países_info'`?

Funciones: Pregunta

Algo muy util es pensar en pseudo código y despues llenar las lineas de codigo (googleando):

Funciones: Pregunta

Algo muy útil es pensar en pseudo código y después llenar las líneas de código (googleando):

```
1 >>> # Hacer un loop para iterar por los países del
    diccionario maestro. Que el loop me de la llave (el país)
    y el valor (el dic interno)
2 >>> #Al diccionario interno crearle un nuevo par de llave y
    valor. Que la llave sea 'gdp_pc' y que el valor sea el
    resultado de la función gdp_per_capita. A la función le
    doy los valores que el diccionario interno tienen
    guardados con las llaves 'gdp' y 'pob'
3
4 >>> paises_info
5 {'argentina': {'cod': 'ARG', 'pob': 44939, 'gdp': 1031215, '
    gdp_pc': 22947}, 'bolivia': {'cod': 'BOL', 'pob': 11513,
    'gdp': 104609, 'gdp_pc': 9086}, 'brasil': {'cod': 'BRA'
    , 'pob': 211050, 'gdp': 3220373, 'gdp_pc': 15259}, '
    chile': {'cod': 'CHL', 'pob': 18952, 'gdp': 476738, '
    gdp_pc': 25155}, 'paraguay': {'cod': 'PRY', 'pob': 7045,
    'gdp': 93062, 'gdp_pc': 13210}, 'uruguay': {'cod': 'URY'
    , 'pob': 3462, 'gdp': 77732, 'gdp_pc': 22453}}
```

Funciones: Ámbito interno vs. ámbito global

Ámbito interno: contiene las variables locales (por ejemplo, de una función)

Ámbito global: contiene las variables globales del módulo actual.

```
1 >>> a = 3 #a definida en ambito global
2 >>>
3 >>> def suma_uno(numero):
4 ...     a = numero + 1 #a definida en ambito interno
5 ...     print(a)
6 >>>
```


Funciones: Ámbito interno vs. ámbito global

Ámbito interno: contiene las variables locales (por ejemplo, de una función)

Ámbito global: contiene las variables globales del módulo actual.

```
1 >>> a = 3 #a definida en ambito global
2 >>>
3 >>> def suma_uno(numero):
4 ...     a = numero + 1 #a definida en ambito interno
5 ...     print(a)
6 >>>
```

```
1 >>> suma_uno(10)
2 11
```

Funciones: Ámbito interno vs. ámbito global

Ámbito interno: contiene las variables locales (por ejemplo, de una función)

Ámbito global: contiene las variables globales del módulo actual.

```
1 >>> a = 3 #a definida en ambito global
2 >>>
3 >>> def suma_uno(numero):
4 ...     a = numero + 1 #a definida en ambito interno
5 ...     print(a)
6 >>>
```

```
1 >>> suma_uno(10)
2 11
```

```
1 >>> print(a)
2 3
```

Es decir, el ámbito interno de la función 'tapa' al ámbito global.

Ámbito: Pregunta

2) Si corremos el siguiente bloque de código ¿Cuántos print() se ejecutarán? ¿Qué valores devolverá cada uno de esos print()? (escribalos en el orden que saldrían)

```
1 >>> gdp_arg = 1031215
2 >>> gdp_u ru = 77732
3 >>> gdp_regional = gdp_arg + gdp_u ru
4
5 >>> def suma_gdp(gdp_regional, gdp_pais):
6 ...     gdp_regional += gdp_pais
7 ...     print(gdp_regional)
8 >>>
9 >>> print(gdp_regional)
10 >>> gdp_pry = 93062
11 >>>
12 >>> suma_gdp(gdp_regional, gdp_pry)
13 >>> print(gdp_regional)
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Manejo de Errores

Al momento de aprender es bueno cometer errores e ir leyendo los motivos de estos errores.

Manejo de Errores

Al momento de aprender es bueno cometer errores e ir leyendo los motivos de estos errores.

```
1 >>> def suma_uno(num):
2     ...     '''
3     ...     Recibe un numero y devuelve ese numero m s uno
4     ...     '''
5     ...     return num + 1
6 >>>
```

```
1 >>> suma_uno(1)
2 2
```

```
1 >>> mi_lista = [1, 20, '30']
2 >>>
3 >>> for elemento in mi_lista:
4     ...     suma_uno(elemento)
5 ?
```

Manejo de Errores

```
1 >>> mi_lista = [1, 20, '30']
2 >>>
3 >>> for elemento in mi_lista:
4 ...     suma_uno(elemento)
5 -----
6 TypeError                                Traceback (most recent call last)
7 <ipython-input-40-aa79ec4ee6f3> in <module>
8     1 for elemento in mi_lista:
9 ----> 2     suma_uno(elemento)
10 <ipython-input-39-adb5a9d88b44> in suma_uno(num)
11     3 def suma_uno(num):
12 ----> 4     return num + 1
13 TypeError: can only concatenate str (not "int") to str
```

Manejo de Errores

Analizar el código por partes, comentar (con #) las líneas con conflicto y usar print() son grandes estrategias en esta etapa.

```
1 >>> for elemento in mi_lista:
2 ...     #suma_uno(elemento)
3 ...     print("item:", elemento)
4 ...     print("tipo:", type(elemento))
```

```
1 item: 1
2 tipo: <class 'int'>
3 item: 2
4 tipo: <class 'int'>
5 item: 30
6 tipo: <class 'str'>
```


Manejo de Errores

Además, en python existen mecanismos para actuar en caso de comportamiento erróneo o inesperado. Para esto, se utiliza la declaración `try/except`:

```
1 >>> for elem in lista:
2 ...     try:
3 ...         suma_uno(elem)
4 ...     except:
5 ...         print("No se pudo sumar", elem, ". Es", type(elem))
```

```
1 >>> suma_uno(mi_lista)
2 2
3 3
4 No se pudo sumar 30. Es <class 'str'>.
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos**
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Módulos

Un módulo es un archivo de Python (con extensión `.py`) que puede contener definiciones de funciones o sentencias ejecutables, y puede ser *importado* desde otros módulos o desde una sesión interactiva (y en particular, desde una sesión de Jupyter).

Un módulo puede ser escrito por el usuario, y también pueden utilizarse módulos escritos por otras personas (previa instalación de los mismos).

Módulos

Un módulo es un archivo de Python (con extensión `.py`) que puede contener definiciones de funciones o sentencias ejecutables, y puede ser *importado* desde otros módulos o desde una sesión interactiva (y en particular, desde una sesión de Jupyter).

Un módulo puede ser escrito por el usuario, y también pueden utilizarse módulos escritos por otras personas (previa instalación de los mismos).

Permiten ampliar notablemente las funcionalidades nativas de Python, y también almacenar código que uno puede querer reutilizar en el futuro (un Jupyter notebook también permite esto último, pero los notebooks no son importables de manera directa).

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- `math`: para funciones matemáticas.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- `math`: para funciones matemáticas.
- `os`: para acceder a funcionalidades del sistema operativo.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- **math**: para funciones matemáticas.
- **os**: para acceder a funcionalidades del sistema operativo.
- **re**: para manejo de expresiones regulares.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- **math**: para funciones matemáticas.
- **os**: para acceder a funcionalidades del sistema operativo.
- **re**: para manejo de expresiones regulares.
- **csv**: para manejo de archivos tabulares separados por comas.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- `math`: para funciones matemáticas.
- `os`: para acceder a funcionalidades del sistema operativo.
- `re`: para manejo de expresiones regulares.
- `csv`: para manejo de archivos tabulares separados por comas.
- `pickle`: para almacenar objetos de Python en el disco, y leerlos.

La biblioteca estándar de Python

Python se distribuye con la *biblioteca estándar* preinstalada. Esta biblioteca contiene un conjunto muy amplio de paquetes para resolver problemas usuales en programación.

Algunos de estos paquetes son:

- **math**: para funciones matemáticas.
- **os**: para acceder a funcionalidades del sistema operativo.
- **re**: para manejo de expresiones regulares.
- **csv**: para manejo de archivos tabulares separados por comas.
- **pickle**: para almacenar objetos de Python en el disco, y leerlos.

La lista completa se puede consultar [acá](#).

¿Cómo importamos un módulo?

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹.

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

```
1 >>> archivo = csv.reader("archivo.csv")
```

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

```
1 >>> archivo = csv.reader("archivo.csv")
```

Se le puede asignar un alias al módulo al momento de importarlo:

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Usando la palabra `import` seguida del nombre del módulo (que es el nombre del archivo *sin* la extensión `.py`)¹. Por ejemplo, importemos el módulo `csv` de la biblioteca estándar

```
1 >>> import csv
```

Esta sentencia genera que ahora podamos utilizar todos los atributos del módulo `csv` de la siguiente forma

```
1 >>> archivo = csv.reader("archivo.csv")
```

Se le puede asignar un alias al módulo al momento de importarlo:

```
1 >>> import csv as comma_separated_values
2 >>> archivo = comma_separated_values.reader("archivo.csv")
```

¹Para módulos definidos por el usuario, por defecto los archivos correspondientes tienen que estar en el directorio de trabajo actual.

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

Es decir, de esta manera *no necesitamos* usar el nombre del módulo (`csv`) para acceder a su atributo (`reader`).

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

Es decir, de esta manera *no necesitamos* usar el nombre del módulo (`csv`) para acceder a su atributo (`reader`).

También se puede usar un alias para el atributo:

¿Cómo importamos un módulo?

Otra forma de importar es usando la sintaxis `from <MODULO> import <ATRIBUTO>`.

```
1 >>> from csv import reader, writer
2 >>> archivo = reader("archivo.csv")
```

Es decir, de esta manera *no necesitamos* usar el nombre del módulo (`csv`) para acceder a su atributo (`reader`).

También se puede usar un alias para el atributo:

```
1 from csv import reader as lector
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos**
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Clases y Objetos

Las *clases* son tipos de datos que permiten representar una entidad encapsulando los datos y funcionalidad asociados a ella. Los *objetos* son instancias de estas clases.

Clases y Objetos

Las *clases* son tipos de datos que permiten representar una entidad encapsulando los datos y funcionalidad asociados a ella. Los *objetos* son instancias de estas clases.

Las clases pueden ser definidas por el usuario, y es la forma de crear nuevos tipos de datos más complejos en base a los tipos nativos de Python.

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

```
1 class Persona():  
2     dni = 57000000  
3     pasaporte = "ABN" + str(dni)
```

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

```
1 class Persona():  
2     dni = 57000000  
3     pasaporte = "ABN" + str(dni)
```

Luego de definir la clase, podemos acceder a sus atributos con un `."`:

Sintaxis de definición de clases

Las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase, seguido de `:`, y luego (bajo indentación) viene una serie de declaraciones:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
```

Luego de definir la clase, podemos acceder a sus atributos con un `."`:

```
1 >>> Persona.dni
2 57000000
3 >>> Persona.pasaporte
4 ABN57000000
```


Clases como "fábricas" de objetos

Las clases permiten definir una función especial llamada *constructor*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def __init__(self, nombre, apellido, edad, sexo):
6         self.nombre = nombre
7         self.apellido = apellido
8         self.edad = edad
9         self.sexo = sexo
```

Clases como "fábricas" de objetos

Las clases permiten definir una función especial llamada *constructor*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def __init__(self, nombre, apellido, edad, sexo):
6         self.nombre = nombre
7         self.apellido = apellido
8         self.edad = edad
9         self.sexo = sexo
```

Una vez definida la clase, el constructor se invoca usando el nombre de la clase y produce un *objeto* de dicha clase:

Clases como "fábricas" de objetos

Las clases permiten definir una función especial llamada *constructor*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def __init__(self, nombre, apellido, edad, sexo):
6         self.nombre = nombre
7         self.apellido = apellido
8         self.edad = edad
9         self.sexo = sexo
```

Una vez definida la clase, el constructor se invoca usando el nombre de la clase y produce un *objeto* de dicha clase:

```
1 >>> juan = Persona("Juan", "Tarufetti", 76, "M")
2 >>> lucia = Persona("Lucia", "Varela", 45, "F")
3
4 >>> lucia.apellido
5 "Varela"
```

Métodos de clases

Las clases permiten no sólo encapsular los datos relativos a un objeto, sino también su comportamiento. Esto se hace a través de funciones, sólo que dentro de una clase se llaman *métodos*:

Métodos de clases

Las clases permiten no sólo encapsular los datos relativos a un objeto, sino también su comportamiento. Esto se hace a través de funciones, sólo que dentro de una clase se llaman *métodos*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def cambiosexo(self, nuevosexo):
6         self.sexo = nuevosexo
7
8     def __init__(self, nombre, apellido, edad, sexo):
9         self.nombre = nombre
10        self.apellido = apellido
11        self.edad = edad
12        self.sexo = sexo
```

Métodos de clases

Las clases permiten no sólo encapsular los datos relativos a un objeto, sino también su comportamiento. Esto se hace a través de funciones, sólo que dentro de una clase se llaman *métodos*:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def cambio_sexo(self, nuevo_sexo):
6         self.sexo = nuevo_sexo
7
8     def __init__(self, nombre, apellido, edad, sexo):
9         self.nombre = nombre
10        self.apellido = apellido
11        self.edad = edad
12        self.sexo = sexo
```

```
1 >>> lucia.cambio_sexo("M")
2 >>> lucia.sexo
3 "M"
```

Ejemplo

Veamos cómo hacer que nuestra clase tenga más sentido para generar los DNI de las personas:

Ejemplo

Veamos cómo hacer que nuestra clase tenga más sentido para generar los DNI de las personas:

```
1 class Persona():
2     dni = 57000000
3     pasaporte = "ABN" + str(dni)
4
5     def cambio_sexo(self, nuevo_sexo):
6         self.sexo = nuevo_sexo
7
8     def __init__(self, nombre, apellido, edad, sexo):
9         self.nombre = nombre
10        self.apellido = apellido
11        self.edad = edad
12        self.sexo = sexo
13        self.dni = Persona.dni
14        Persona.dni += 1
```


Ejemplo

Ahora veamos si funciona el creador de DNI consecutivos con personas recién nacidas:

Ejemplo

Ahora veamos si funciona el creador de DNI consecutivos con personas recién nacidas:

```
1 >>> mateo = Persona("Mateo", "Michel", 0, "M")
2 >>> sofia = Persona("Sofia", "Torino", 0, "F")
3 >>> axel = Persona("Axel", "Lopez", 0, "M")
```

Ejemplo

Ahora veamos si funciona el creador de DNI consecutivos con personas recién nacidas:

```
1 >>> mateo = Persona("Mateo", "Michel", 0, "M")
2 >>> sofia = Persona("Sofia", "Torino", 0, "F")
3 >>> axel = Persona("Axel", "Lopez", 0, "M")
```

```
1 >>> mateo.dni
2 57000000
3 >>> sofia.dni
4 57000001
5 >>> axel.dni
6 57000002
7 >>> mateo.pasaporte
8 ANB57000000
9 >>> sofia.pasaporte
10 ANB57000001
```

Herencia

El mecanismo de *herencia* de Python nos permite definir clases derivadas que extienden o modifican el comportamiento de la clase base. La sintaxis para definir una subclase es:

Herencia

El mecanismo de *herencia* de Python nos permite definir clases derivadas que extienden o modifican el comportamiento de la clase base. La sintaxis para definir una subclase es:

```
1 class SubClase( ClaseBase ) :  
2     ...
```

Herencia

El mecanismo de *herencia* de Python nos permite definir clases derivadas que extienden o modifican el comportamiento de la clase base. La sintaxis para definir una subclase es:

```
1 class SubClase( ClaseBase ):
2     ...
```

Por ejemplo,

```
1 class Estudiante(Persona):
2     def __init__(self, nombre, apellido, ..., carrera,\
materias_aprobadas):
3         super().__init__(nombre, apellido, ...)
4         self.carrera = carrera
5         self.materias_aprobadas = materias_aprobadas
6
7     def aprobar(nueva_materia):
8         self.materias_aprobadas.append(nueva_materia)
```

Pregunta

Si definimos la siguiente clase:

```
1 class Persona:
2     def __init__(self, nombre, apellido, edad):
3         self.nombre = nombre
4         self.apellido = apellido
5         self.edad = edad
6
7     def __str__(self):
8         return self.nombre + " " + self.apellido
```

Pregunta

Si definimos la siguiente clase:

```
1 class Persona:
2     def __init__(self, nombre, apellido, edad):
3         self.nombre = nombre
4         self.apellido = apellido
5         self.edad = edad
6
7     def __str__(self):
8         return self.nombre + " " + self.apellido
```

3) ¿Qué imprimiría el siguiente código:

```
1 >>> juan = Persona("Juan", "Tarufetti", 74)
2 >>> print(juan)
```

4) ¿Cómo agregarían un metodo llamado `cambio_nombre` que actualice el nombre de la persona que tramita un cambio de nombre?

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "w")
2 >>> f.write("Estas seran las primeras palabras en el archivo
. \n")
3 >>> f.close()
```

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "w")
2 >>> f.write("Estas seran las primeras palabras en el archivo
. \n")
3 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "a")
2 >>> f.write("Estas palabras se agregaran al final del
archivo")
3 >>> f.close()
```

Entrada de y salida a archivos txt

```
1 >>> f = open("archivo.txt")
2 >>> print(f.read())
3 HOLA A TODOS
4 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "w")
2 >>> f.write("Estas seran las primeras palabras en el archivo
. \n")
3 >>> f.close()
```

```
1 >>> f = open("archivo.txt", "a")
2 >>> f.write("Estas palabras se agregaran al final del
archivo")
3 >>> f.close()
```

```
1 >>> with open("archivo.txt") as f:
2 >>>     print(f.read())
3 Estas seran las primeras palabras en el archivo.
4 Estas palabras se agregaran al final del archivo
```

Temario de la clase

- 1 Funciones
- 2 Manejo de Errores
- 3 Módulos
- 4 Clases y objetos
- 5 Importar/Exportar archivos txt
- 6 Importar/Exportar archivos de tablas

Instalación e introducción a pandas

Para poder seguir esta parte de la clase a la par por favor instalen la librería **pandas**

Instalación e introducción a pandas

Para poder seguir esta parte de la clase a la par por favor instalen la librería **pandas**

En la consola:

```
1 >>> pip install pandas
```

O en el Jupyter Notebook:

```
1 import sys
2 !{sys.executable} -m pip install pandas
```


Instalación e introducción a pandas

Para poder seguir esta parte de la clase a la par por favor instalen la librería **pandas**

En la consola:

```
1 >>> pip install pandas
```

O en el Jupyter Notebook:

```
1 import sys
2 !{sys.executable} -m pip install pandas
```

pandas nos permitirá trabajar con datos en formato estructurados. Es decir con tablas de dato como las que estamos acostumbrados a trabajar en Excel o Stata, que contienen la información organizada en filas y columnas.

Importar pandas

pandas es la librería más usada para este tipo de operaciones, tiene funciones y métodos que facilitan mucho el trabajo con tablas de datos (*dataframes*).

En esta clase solo les voy a mostrar como usarla para:

- importar archivos de distintos formatos;
- explorar si se importaron los datos del archivo;

Importar pandas

pandas es la librería más usada para este tipo de operaciones, tiene funciones y métodos que facilitan mucho el trabajo con tablas de datos (*dataframes*).

En esta clase solo les voy a mostrar como usarla para:

- importar archivos de distintos formatos;
- explorar si se importaron los datos del archivo;

Como primer paso, importemos la librería:

```
1 >>> import pandas as pd
```

Abrir un dataframe I

Primer paso, abramos un archivo. Ya vimos una forma de abrir txt, ahora vamos a abrir una tabla con Pandas.

Abrir un dataframe I

Primer paso, abramos un archivo. Ya vimos una forma de abrir txt, ahora vamos a abrir una tabla con Pandas.

Por convención:

- pandas se importa con el alias "pd"
- la base importada se refiere como dataframe, y se abrevia df

Abrir un dataframe I

Primer paso, abramos un archivo. Ya vimos una forma de abrir txt, ahora vamos a abrir una tabla con Pandas.

Por convención:

- pandas se importa con el alias "pd"
- la base importada se refiere como dataframe, y se abrevia df

Pandas tiene funciones para abrir muchos tipos de archivos estructurados. Algunos ejemplos son:

- `pd.read_excel()`
- `pd.read_csv()`
- `pd.read_stata()`
- `pd.read_spss()`

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel('file_path.xlsx')
```

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel('file_path.xlsx')
```

Por default Pandas abre la primera hoja del archivo (esté visible o escondida). Para indicar otra hoja:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```


Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel('file_path.xlsx')
```

Por default Pandas abre la primera hoja del archivo (esté visible o escondida). Para indicar otra hoja:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Ahora, esa función abre el archivo pero no lo guarda en memoria. Para eso hay que asignarle un nombre de variable:

```
1 >>> import pandas as pd
2 >>> df = pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv('file_path.csv')
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv('file_path.csv')
```

Al abrir un csv, python intenta inferir el separador. El separador es el símbolo que separa los valores: ",", ";", etc. En general funciona bien, si hay algún inconveniente se puede especificar con:

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv('file_path.csv', sep=";")
```

Abrir un dataframe III

Abrir un .csv es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv('file_path.csv')
```

Al abrir un csv, python intenta inferir el separador. El separador es el símbolo que separa los valores: ",", ";", etc. En general funciona bien, si hay algún inconveniente se puede especificar con:

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv('file_path.csv', sep=";")
```

Abrir un archivo de Stata es similar.

```
1 >>> import pandas as pd
2 >>> df = pd.read_stata('file_path.dta')
```

Nota: Pandas tiene documentación muy completa.

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Para hacer una primera visualización de los datos importados podemos usar:

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Para hacer una primera visualización de los datos importados podemos usar:

- `df.head(N)`: nos muestra las primeras N líneas
- `df.tail(N)`: nos muestra las últimas N líneas
- `df.sample(N)`: muestra aleatoria de N líneas

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Para hacer una primera visualización de los datos importados podemos usar:

- `df.head(N)`: nos muestra las primeras N líneas
- `df.tail(N)`: nos muestra las últimas N líneas
- `df.sample(N)`: muestra aleatoria de N líneas

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv("file_path.csv")
3 >>> df.head(5) #imprime nombres de las columnas y las 5
               primeras lineas
```

```
1 >>> df.tail(2) #igual que head pero empieza desde el fondo
```

```
1 >>> df.sample(10) #muestra aleatoria de n=10
```

Python para Economistas

Universidad Nacional de La Plata

2 de octubre de 2022

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: append y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Temario de la clase

1 Repaso de la última clase

2 Objetivos de la clase

- Arrays
- Operaciones con arrays
- Números aleatorios

3 Pandas

- Formato estructurado: abrir y explorar
- Dataframes: operaciones y aplicación de funciones
- Dataframes: seleccionar sub-conjuntos
- Dataframes: append y merge
- Dataframes: aggregate

4 Matplotlib

- Introducción a Matplotlib
- Ejemplo práctico

En la clase pasada vimos:

- 1 Funciones
- 2 Manejo de errores
- 3 Módulos
- 4 Clases y objetos
- 5 Manejo básico de archivos

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: append y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Objetivos de la clase

La clase de esta semana se centra en tres librerías centrales de Python:

- **NumPy** es una librería de cálculo numérico para Python, muchas de las principales librerías en análisis de datos están construidas sobre ella.
- **Pandas** es quizás la librería que más usen en sus primeros pasos en Python. Sirve para el trabajo con tablas de datos estructuradas, importar tablas de información, manipularlas y analizarlas.
- **Matplotlib**, la librería de base para graficar.

Hay mucho más para conocer sobre estas librerías de lo que se cubre en esta clase. El objetivo es que tengan el conocimiento suficiente para comenzar a usarlas y resolver las tareas más comunes.

Introducción a NumPy

NumPy es una librería de cálculo numérico para Python, lanzada inicialmente en 1995. Está escrita en lenguajes de bajo nivel como C y Fortran para permitir realizar operaciones matemáticas de manera eficiente.

Muchas librerías muy utilizadas en análisis de datos tienen a NumPy como librería base, y agregan funcionalidades adicionales por encima de ella. Algunos ejemplos son:

- Pandas: para manejo de datos tabulares,
- Matplotlib: para gráficos,
- SciPy: para cálculo científico,
- scikit-learn: para aprendizaje automático,
- TensorFlow y PyTorch: para aprendizaje profundo.

Las funcionalidades de esta librería que estudiaremos son:

- Arrays multidimensionales
- Operaciones numéricas básicas
- Operaciones de álgebra lineal
- Generación de números aleatorios

Como primer paso, importemos la librería (lo vimos en Clase 2):

```
1 >>> # primero importemos la libreria
2 >>> import numpy as np
3 >>> # definamos decimales en los resultados imprimidos
4 >>> np.set_printoptions(precision=3)
```

Arrays

Numpy está construido para trabajar de manera eficiente con vectores y matrices. Si bien un vector puede guardar información de manera similar que una lista, lo hace de manera más eficiente. Esta eficiencia en el almacenamiento y procesamiento de datos la hace más adecuada para procesamientos grandes.

```
1 >>> vector = np.array((1.,2,3,4))
2 >>> print(vector)
3 [ 1.  2.  3.  4.]
4 >>> vector.dtype
5 dtype('float64')
6 >>> print("vector ndim: ", vector.ndim)
7 vector ndim:  1
8 >>> print("vector shape:", vector.shape)
9 vector shape: (4,)
10 >>> print("vector size: ", vector.size)
11 vector size:  4
```


Arrays

Sigamos con otros ejemplos:

```
1 >>> matriz = np.array([[1,2,3], [4,5,6]])
2 >>> print(matriz)
3 [[1 2 3]
4  [4 5 6]]
5 >>> print("matriz ndim: ", matriz.ndim)
6 matriz ndim: 2
7 >>> print("matriz shape:", matriz.shape)
8 matriz shape: (2, 3)
9 >>> print("matriz size: ", matriz.size)
10 matriz size: 6
```

Arrays

Y para el último ejemplo, creamos un **Tensor**. Un tensor es una grilla de números, arrays, de n dimensiones. Un vector es un tensor de rango 1. Una matriz es un tensor de rango 2.

```
1 >>> # definimos un tensor de rango 3
2 >>> tensor3 = np.array([[[[-1,-2], [-3,-4]], [[0,0],
3 [[0,0]], [[2,1], [1,5]]]])
4 >>> print(tensor3)
5 [[[-1 -2]
6      [-3 -4]]
7      [[ 0  0]
8      [ 0  0]]
9      [[ 2  1]
10     [ 1  5]]]
11 >>> print("tensor3 ndim: ", tensor3.ndim)
12 tensor3 ndim:  3
13 >>> print("tensor3 shape:", tensor3.shape)
14 tensor3 shape: (3, 2, 2)
15 >>> print("tensor3 size: ", tensor3.size)
16 tensor3 size:  12
```

Operaciones on arrays

Similar a lo que vimos con operaciones entre variables, con Numpy es posible hacer operaciones entre arrays.

A continuación listamos algunos ejemplos:

```
1 >>> # creamos dos arrays
2 >>> a = np.array([1, 2, 3, 4],[1, 2, 3, 4])
3 >>> b = np.array([4, 2, 2, 4],[4, 2, 2, 4])
4 >>> # podemos realizar operaciones entre matrices
5 >>> a + b
6 >>> a - b
7 >>> a * b
8 >>> a / b
9 >>> a ** b
10 >>> a > 0.5
11 >>> 2 * a + 3
```

Números aleatorios

Por último, Numpy contiene un submódulo útil para crear números pseudo aleatorios.

```
1  >>> # importamos el submodulo
2  >>> from numpy import random
3  >>> # creamos un entero aleatorio entre 0 y 1000
4  >>> x = random.randint(1000)
5  >>> # creamos un float aleatorio entre 0 y 1
6  >>> y = random.rand()
7  >>> # inspeccionamos
8  >>> print(x, y)
9  >>> # creamos matrices aleatorias
10 >>> a = random.rand(2,3)
11 >>> print(a)
```

Preguntas para responder en el cuestionario que acompaña la presentación. Para poder contestar estas preguntas van a tener que explorar la documentación del submódulo random:

1. ¿Cual el código correcto para crear un vector de 5 números enteros aleatorios entre 0 y 100? (ver opciones en cuestionario)
2. ¿Cual el código correcto para crear un matriz de 2x3 conformada por números enteros aleatorios entre 0 y 100? (ver opciones en cuestionario)

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: append y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Formato estructurado

Objetivo: Trabajar con datos en formato estructurado. Refiere a tablas de datos como las que estamos acostumbrados a trabajar en Excel o Stata, información organizada en filas y columnas.

Pandas: es la librería más usada para este tipo de operaciones, tiene funciones y métodos que facilitan mucho el trabajo con datos tabulares.

Pandas define dos clases fundamentales:

- **Series:** similar a un `np.ndarray` unidimensional, pero los elementos tienen un índice asociado.
- **DataFrame:** la "tabla". Técnicamente, es una secuencia de **Series** donde el índice está compartido. En otras palabras, cada **Series** es una columna de la tabla, y el índice es un identificador para la fila.

En esta clase vamos a usar Pandas para:

- importar/exportar archivos de distintos formatos;
- explorar la tabla de datos;
- filtrar datos;
- agregar columnas / filas;
- unir bases (`merge`, `join`).

Abrir un dataframe I

Como primer paso, importemos la librería:

```
1 >>> import pandas as pd
```

Empecemos por abrir un archivo. Ya vimos una forma de hacerlo en la clase 2, ahora vamos a profundizar.

Por convención:

- Pandas se importa con el alias `pd`,
- la base importada se refiere como dataframe, y se abrevia `df`.

Pandas tiene funciones para abrir muchos tipos de archivos estructurados. Algunos ejemplos son:

- `pd.read_excel()`
- `pd.read_csv()`
- `pd.read_stata()`
- `pd.read_spss()`

Abrir un dataframe II

Veamos en detalle. Primero abrir un archivo Excel, para hacerlo:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx")
```

Por defecto, Pandas abre la primera hoja del archivo (esté visible o escondida). Para indicar otra hoja:

```
1 >>> import pandas as pd
2 >>> pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Ahora, esa función abre el archivo pero no lo guarda en memoria. Para eso hay que asignarle un nombre de variable:

```
1 >>> import pandas as pd
2 >>> df = pd.read_excel("file_path.xlsx", sheet_name="Hoja2")
```

Abrir un dataframe III

Abrir un `.csv` es prácticamente lo mismo:

```
1 >>> import pandas as pd # importar una vez al comienzo de la
    sesion
2 >>> df = pd.read_csv("file_path.csv")
```

Al abrir un csv, Python intenta inferir el separador. El separador es el símbolo que separa los valores: `","`, `","`, `","`, etc. En general funciona bien, si hay algún inconveniente se puede especificar con:

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv("file_path.csv", sep=";")
```

Abrir un archivo de Stata es similar.

```
1 >>> import pandas as pd
2 >>> df = pd.read_stata("file_path.dta")
```

Nota: Pandas tiene documentación muy completa.

Crear un DataFrame

Veamos como crear un data frame vacío para luego agregarle filas.

```
1 >>> import pandas as pd
2 >>> df = pd.DataFrame(columns=["Nombre", "Apellido", "Curso",
    , "Profesor"])
```

Para agregarle contenido a mi tabla una opción sería agregarle una lista con los cuatro valores correspondientes a cada columna

```
1 >>> profesor1 = ["Belen", "Michel", "Python", True]
2 >>> profesor2 = ["Rodrigo", "Bonazzola", "Python", True]
3 >>> alumno = ["Juan", "Tarufetti", "Python", False]
```

```
1 >>> df.loc[0] = profesor1
2 >>> df.loc[1] = profesor2
3 >>> df.loc[len(df)] = alumno
```

Explorar los datos I

¿Cómo exploramos los datos que importamos? Hay muchas maneras. Primero veamos las funciones de pandas que sirven para hacer una inspección general y una primera visualización de los datos importados:

- `df.head(N)`: nos muestra las primeras N líneas
- `df.tail(N)`: nos muestra las últimas N líneas
- `df.sample(N)`: muestra aleatoria de N líneas

```
1 >>> import pandas as pd
2 >>> df = pd.read_csv("file_path.csv")
3 >>> df.head(5) # imprime nombres de las columnas y las 5
    primeras lineas
4 < OUTPUT SUPRIMIDO >
5 >>> df.tail(2) # igual que head pero empieza desde el fondo
6 < OUTPUT SUPRIMIDO >
7 >>> df.sample(10) #muestra aleatoria de n=10
8 < OUTPUT SUPRIMIDO >
```

Explorar los datos II

En los archivos de la Clase 3 (GitHub) hay un archivo `tabla_ejemplo.xlsx` que incluye registros inventados para grupos de estudiantes. Para cada grupo se cuenta con un identificador, la cantidad de integrantes y la edad promedio.

```
1 >>> import pandas as pd
2 >>> df = pd.read_excel("tabla_ejemplo.xlsx")
3 >>> df.head(2)
4      grupo  integrantes  edad_promedio
5 0        A           41       25.390012
6 1        B           55       39.969692
7 >>> # los 0 y 1 sobre el margen izquierdo son el numero de
      fila
8 >>> df.sample(3)
9      grupo  integrantes  edad_promedio
10 11      L           22       33.753149
11 8        I           52       25.543545
12 14      Z           50       36.578090
```

Explorar los datos III

Al importar un df, es una buena práctica imprimir algunas líneas para verificar que se haya cargado bien. También es útil imprimir el listado de nombres de las columnas y el tipo de dato:

- `df.columns` - listado de nombres de columnas,
- `df.dtypes` - tipo de dato por columna,
- `df.info` - información más detallada sobre el DataFrame.

```
1 >>> #continuando con el df ya definido
2 >>> df.columns
3 Index(['grupo', 'integrantes', 'edad_promedio'], dtype='
      object')
4 >>> df.dtypes
5 grupo                object
6 integrantes          int64
7 edad_promedio       float64
8 dtype: object
```


Explorar los datos IV

Y para ver en más detalle:

```
1 >>> df.info(verbose=True) # verbose imprime resumen completo
2 <class 'pandas.core.frame.DataFrame'>
3 RangeIndex: 15 entries, 0 to 14
4 Data columns (total 3 columns):
5 grupo                15 non-null object
6 integrantes           15 non-null int64
7 edad_promedio        15 non-null float64
8 dtypes: float64(1), int64(1), object(1)
9 memory usage: 440.0+ bytes
```

Exportar I

Exportar también es sencillo con Pandas. Dos opciones comunes son:

- `df.to_excel()`
- `df.to_csv()`

Por default, el dataframe exportado se guardará en directorio en el que se está trabajando, con el nombre de archivo que especifiquemos.

```
1 >>> df.to_excel("exportar_ejemplo.xlsx", index=False)
2 >>> # listo!
```

Exportar II

Veamos si funcionó.

```
1 >>> df = pd.read_excel("exportar_ejemplo.xlsx")
2 >>> df.sample(1)
3      grupo  integrantes  edad_promedio
4 2      "C"           36         31.672932
5 >>> # Funciona! Ahora exportamos a csv
6 >>> # tambien podemos definir el separador
7 >>> df.to_csv("exportar_ejemplo.csv", index=False, sep="|")
8 >>> df = pd.read_csv("exportar_ejemplo.csv")
9 >>> df.columns
10 Index(['grupo|integrantes|edad_promedio'], dtype='object')
11 >>> # Una columna? Porque no especificamos el separador.
12 >>> df = pd.read_csv("exportar_ejemplo.csv" , sep="|")
13 >>> df.columns
14 Index(['grupo', 'integrantes', 'edad_promedio'], dtype='object')
```

Preguntas para responder en el cuestionario que acompaña la presentación:

3. ¿Cuál es la manera correcta de abrir un archivo .xlsx? (ver opciones en cuestionario)
4. ¿Qué código escribimos para saber el tipo de dato de una o varias columnas?

Trabajando con dataframes I

A continuación vamos a ver una serie de acciones que podemos realizar con dataframes.

- crear columnas
- cambiar tipos de datos
- aplicar funciones
- filtrar (slice)
- eliminar duplicados
- agregar (append)
- join/merge
- agrupar (aggregate)

Trabajando con dataframes II

Empecemos con cómo seleccionar columnas. Podemos hacerlo de dos maneras:

- `df["nombre_columna"]`: esto devuelve la columna como un objeto de tipo `pd.Series`. Se puede usar con nombres que incluyen espacios, sirve para crear columnas y operaciones con y entre columnas existentes. Es la que vamos a usar.
- `df.nombre_columna`: acceder como atributo de la dataframe. Una limitación es que no se puede usar con nombres que incluyen espacios, ni tampoco para crear columnas nuevas. Sí para operaciones con y entre columnas existentes.

Operaciones con columnas I

Un ejemplo:

```
1 >>> df = pd.read_excel("tabla_ejemplo_2.xlsx") #ejemplo2
2 >>> df.columns
3 Index(['grupo', 'inscriptos_ronda1', 'inscriptos_ronda2',
4        'edad_promedio', 'asignatura', 'area'], dtype='object')
5 >>> # df (nombre de la tabla) + enter, les imprime una
      muestra.
6 >>> # si df es grande, muestra las filas/columnas de los
      extremos, indicando con "..." las filas/columnas
      omitidas.
7 >>> # prueben poniendo df['asignatura'] y enter para
      visualizar
8 >>> # creamos una columna nueva con total de inscriptos
      llamada "inscriptos_total"
9 >>> df["inscriptos_total"] = df["inscriptos_ronda1"] + df["
      inscriptos_ronda2"]
10 >>> # impriman df o df["inscriptos_total"] para ver el
      resultado
```

Operaciones con columnas II

Continúa:

```
1 >>> # sumemos 2 inscriptos a cada grupo de la ronda 2 y
    volvamos a calcular el total
2 >>> df["inscriptos_ronda2"] = df["inscriptos_ronda2"] + 2
3 >>> df["inscriptos_total"] = df["inscriptos_ronda1"] + df["
    inscriptos_ronda2"]
4 >>> # generemos un id concatenando area y materia
5 >>> df["area_asignatura"] = df["area"] + df["asignatura"]
6 >>> # impriman el resultado
7 >>> # concatenado asi es confuso, incluyamos un separador
8 >>> df["area_asignatura"] = df["area"]+"_"+df["asignatura"]
9 >>> # impriman el resultado
10 >>> # generamos una columna que indique estado de
    inscripcion
11 >>> df["inscripcion_estado"] = "CERRADA"
12 >>> # impriman el resultado
```


Tipos de datos

Ya vimos cómo podemos usar Pandas para conocer el tipo de dato de cada columna, usando `df.info(verbose=True)`.

Para ver el tipo de dato de una columna podemos usar `df['columna'].dtype`:

```
1 >>> # inspeccionar el tipo de dato de una columna
2 >>> df["inscriptos_total"].dtype # singular, dtype != dtypes
3 dtype('int64')
4 >>> # con .astype() casteamos a otro formato
5 >>> df["inscriptos_total"] = df["inscriptos_total"].astype(
6     str)
7 >>> df["inscriptos_total"].dtype
8 dtype('O')
9 >>> # "O" de "object"
10 >>> # volvemos a formato int
11 >>> df["inscriptos_total"] = df["inscriptos_total"].astype(
12     int)
```

Aplicar funciones a columnas I

Las columnas son objetos iterables sobre los que podemos aplicar una función y afectará los registros en cada uno de los elementos del array.

Podemos aplicar funciones predefinidas como también funciones propias. Un ejemplo fue la función `df['columna'].astype()`. Otro ejemplo.

```
1 >>> # la funcion round() redondea al entero mas cercano
2 >>> df['edad_promedio_r'] = round(df['edad_promedio'])
```

También podemos usar `df['columna'].apply(funcion_x)`, donde `apply()` inserta `df['columna']` como parametro en `funcion_x()`. `apply` itera sobre los inputs y aplica la función sobre cada elemento.

```
1 >>> # usando la misma funcion
2 >>> df["edad_promedio_r"] = df['edad_promedio'].apply(round)
```

Una función puede no correr depende de si el input esperado es una **Series** o un elemento de la **Series**. Con `.apply()` funciona de ambas maneras.

Aplicar funciones a columnas II

Ahora un ejemplo con una función propia. Primero definimos la función:

```
1 >>> # function simple
2 >>> def clasificar_tamano(inscriptos):
3 ...     if inscriptos < 30:
4 ...         tamano = "chico"
5 ...     else:
6 ...         tamano = "grande"
7 ...     return tamano
```

Ahora la aplicamos:

```
1 >>> df['inscriptos_total'].apply(clasificar_tamano)
2 >>> df["tamano_clase"] = df['inscriptos_total'].apply(
    clasificar_tamano)
```

Seleccionar columnas

Para seleccionar un sub-conjunto de filas y/o columnas hay distintas alternativas. Para seleccionar columnas:

```
1 >>> # ya vimos como seleccionar una columna
2 >>> df["grupo"]
3 < OUTPUT SUPRIMIDO >
4 >>> # para seleccionar varias pasamos una lista
5 >>> df[["grupo","materia","inscriptos_total"]]
6 < OUTPUT SUPRIMIDO >
7 >>> # podemos guardar una copia como otro dataframe
8 >>> df_resumen = df[["grupo","inscriptos_total"]].copy()
```

Seleccionar filas I

Para seleccionar un sub-conjunto de filas podemos usar los índices:

```
1 >>> # para ver las filas 3 a 5
2 >>> df[3:6] # el limite superior no se incluye
3 >>> # para ver las primeras 3 filas
4 >>> df[:3]
5 < OUTPUT SUPRIMIDO >
6 >>> # para ver las ultimas 3 filas
7 >>> df[-3:]
8 < OUTPUT SUPRIMIDO >
```

También podemos usar condiciones:

```
1 >>> # slicing
2 >>> df[df["inscriptos_total"]<30]
3 < OUTPUT SUPRIMIDO >
4 >>> # y combinar condiciones
5 >>> df[(df["inscriptos_total"]<30)&(df["edad_promedio"]>40)]
6 < OUTPUT SUPRIMIDO >
```

Seleccionar filas II

Para filtrar usando condiciones:

- Si hay una condición: `df[condición]`
- Si hay más de una condición: `df[(condición1) & (condición2)]` o `df[(condición1) | (condición2)]`

Dónde la condición es booleana (evalúa a True o False), de ser 2 o más, debe estar unida por un operador `&` o `|` (para AND o OR respectivamente). Si son más de dos condiciones presten atención a los paréntesis. Algo para tener en cuenta:

```
1 >>> # and!=& y or!=|, no son intercambiables:
2 >>> # esto:
3 >>> df[(condicion1)and(condicion2)] # incorrecto
4 >>> # NO es igual a esto:
5 >>> df[(condicion1)&(condicion2)] # correcto
6 >>> # y esto:
7 >>> df[(condicion1)or(condicion2)] # incorrecto
8 >>> # TAMPOCO es igual a esto:
9 >>> df[(condicion1)|(condicion2)] # correcto
```

Seleccionar filas y columnas

Para combinar filtros de filas y columnas:

- `df.loc[filas,columnas]`: usando el índice para identificar. Notar que a diferencia de lo que ocurre con slicing, al usar `loc` con enteros el límite superior SI se incluye.
- `df.iloc[filas,columnas]`: usando la posición (como entero) para identificar. En `.iloc` el límite superior NO se incluye.

Nota: muchas veces el label e index de una fila son iguales (numéricos).

```
1 >>> # ejemplo .loc
2 >>> df.loc[3:4,["grupo","asignatura"]]
3     grupo asignatura
4 3      D    Historia
5 4      E      Fisica
6 >>> # ejemplo .iloc:
7 >>> df.iloc[3:5,[0,4]]
8     grupo asignatura
9 3      D    Historia
10 4      E      Fisica
```

Eliminar filas y columnas

Para eliminar filas y columnas podemos:

- 'pisar' el dataframe con la selección de las filas/columnas que SI queremos guardado una nueva copia sobre la anterior.
- usar la función `df.drop()`.

La función `.drop()` toma como argumento una lista de labels, y puede usarse para eliminar filas o columnas. Con el parametro 'axis' definimos si eliminar filas (`axis=0`, el default) o columnas (`axis=1`).

```
1 >>> # ejemplo eliminar filas
2 >>> df.drop([0,3,12]) # implicitamente llama axis=0
3 < OUTPUT SUPRIMIDO >
4 >>> # ejemplo eliminar columnas
5 >>> df.drop(["inscripcion_estado"],axis=1) # no guarda
6 < OUTPUT SUPRIMIDO >
7 >>> # df = df.drop(["inscripcion_estado"],axis=1)
8 >>> # o definir el parametro inplace=True
9 >>> df.drop(["inscripcion_estado"], axis=1,inplace=True)
```


Preguntas

Preguntas para responder en el cuestionario que acompaña la presentación:

5. ¿Cómo se cambia el tipo de dato de una columna? (ver opciones en cuestionario)
6. ¿Qué resultado arroja el código `df[ df['inscriptos_total'] <30 ]` ?

Append

Una ventaja de Python es la facilidad con la que se puede trabajar con multiples bases al mismo tiempo. Si tenemos un conjunto de bases que queremos agregar una debajo de la otra (EPH por ejemplo), podemos usar `df_a.append(df_b)`

```
1 >>> # cargemos una versi n recortada de la base
2 >>> df_a = pd.read_excel("tabla_ejemplo_3a.xlsx") #exploren
      contenido
3 >>> len(df_a)
4 9
5 >>> df_b = pd.read_excel("tabla_ejemplo_3b.xlsx") #exploren
      contenido
6 >>> len(df_b)
7 9
8 >>> df = df_a.append(df_b) # exploren para ver resultado
9 >>> len(df)
10 18
```

Merge I

Podemos usar las funciones `.join()` y `.merge()` para unir horizontalmente bases con un identificador común. Veamos el caso de `.merge()` (`join` es similar, lo pueden ver en la documentación de Pandas).

La función `merge` se aplica con la forma:

`df_a.merge(df_b,on=lista_columnas_id, how=tipo_join)`. Donde los tipos de `join` son:

- **left:** preserva todos los datos de la base izquierda y sólo los `matcheados` de la base derecha.
- **right:** preserva todos los datos de la base derecha y sólo los `matcheados` de la base izquierda.
- **outer:** preserva todos los datos (`matcheados` o no)
- **inner:** preserva los datos de filas con identificador común (`matcheados`).

Merge II

Retomando el ejemplo:

```
1 >>> # continuando con el ejercicio
2 >>> df.columns
3 Index(['grupo', 'inscriptos_ronda1', 'inscriptos_ronda2', '
    edad_promedio'], dtype='object')
4 >>> df_c = pd.read_excel("tabla_ejemplo_3c.xlsx") #exploren
    contenido
5 >>> df_c.columns
6 Index(['grupo', 'asignatura', 'area'], dtype='object')
7 >>> df = df.merge(df_c) # exploren para ver resultado
8 >>> df.columns
9 Index(['grupo', 'inscriptos_ronda1', 'inscriptos_ronda2', '
    edad_promedio', 'asignatura', 'area'], dtype='object')
```

Datos duplicados y reset index

Es una buena práctica inspeccionar los resultados después de hacer un append o merge. Por ejemplo, luego del append, la indexación de las filas quedó repetida ya que se preservaron los índices de las bases originales. Para arreglar esto:

```
1 >>> # luego del .append()
2 >>> df.reset_index(drop=True) # index original es descartado
```

Dependiendo de los datos que estemos usando, o si las bases no están bien curadas, realizar un merge puede generar duplicación de registros. En este caso lo simulamos. Para eliminar registros duplicados:

```
1 >>> df.tail(6)
2 >>> df.drop_duplicates(inplace=True)
3 >>> # como quedo el indice de las filas?
4 >>> df.reset_index(drop=True, inplace=True)
5 >>> # inplace guarda el resultado
6 >>> df
7 < OUTPUT SUPRIMIDO >
```

Aggregate I

Una operación frecuente con dataframes es agregar los datos, agrupando sobre un conjunto de variables y aplicando una función de agregación a otras. Con pandas esto se logra usando la función `.groupby()`, de la forma:

```
df.groupby(by=lista_columnas_agrupamiento).agg(dict_var_func)
```

Dónde `lista_columnas_agrupamiento` es una lista con los nombres de columnas sobre las que se agrupa, y `dict_var_func` es un diccionario de variable a agregar (llave) y la función con la que se agrega (valor).

Algunas de las funciones de agregación son:

- `sum`
- `mean`
- `count`
- `first` / `last`
- `min` / `max`

Aggregate II

Una operación

```
1 >>> # seguimos con el mismo df ya definido
2 >>> df_agg = df.groupby(by=["area","asignatura"]).agg({"
    inscriptos_ronda1":"sum","inscriptos_ronda2":"sum","
    edad_promedio":"mean"})
3 >>> # las variables de agrupamiento pasan a definir el
    indice
4 >>> df_agg.reset_index(inplace=True) # drop=False por
    default
5 >>> df_agg
6 < OUTPUT SUPRIMIDO >
```

Preguntas para responder en el cuestionario que acompaña la presentación:

7. Si no definimos el tipo de join, ¿cual se ejecuta por default?

Nota: Esto no se puede inferir de los resultados del ejemplo, buscar en la documentación de Pandas (`pandas.DataFrame.join`).

8. Si hacemos `df_a.append(df_b)`, dónde `df_b` tiene algunas columnas en común y otras distintas a `df_a`, ¿qué pasará?

Nota: no está mencionado en el video, buscar en la la documentación de Pandas (`pandas.DataFrame.append`).

Temario de la clase

- 1 Repaso de la última clase
- 2 Objetivos de la clase
 - Arrays
 - Operaciones con arrays
 - Números aleatorios
- 3 Pandas
 - Formato estructurado: abrir y explorar
 - Dataframes: operaciones y aplicación de funciones
 - Dataframes: seleccionar sub-conjuntos
 - Dataframes: append y merge
 - Dataframes: aggregate
- 4 Matplotlib
 - Introducción a Matplotlib
 - Ejemplo práctico

Introducción a Matplotlib I

Matplotlib es la librería base de graficación, sobre la cual se montan otras librerías (como wrappers). Dentro de Matplotlib, usamos la dependencia "pyplot" que se instala con la librería. Por convención importamos así:

```
1 >>> # importar libreria grafica
2 >>> import matplotlib.pyplot as plt
3 >>> #"plt" siendo el nombre de convencion
```

Matplotlib genera los gráficos sobre dos objetos interrelacionados:

- **Figure:** la hoja en blanco, el recuadro que contiene el/los gráfico/s. En términos prácticos esto es ocurre detrás de escenas, pero es lo que permite dibujar el gráfico.
- **Axes:** el gráfico en si, los ejes y la información gráfica. La representación de la información sobre ejes.

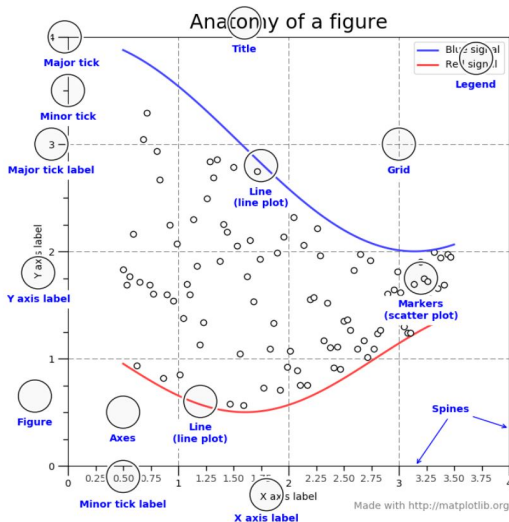
Introducción a Matplotlib II

¿Que podemos definir en un gráfico? Casi todo. Lo más que aprendamos lo más específicos que podemos ser (similar a Stata pero más flexible y con resultados superadores).

[Ver en la galería de ejemplos de Matplotlib](#)

Las partes de un gráfico:

Introducción a Matplotlib II



Fuente: Matplotlib User's Guide

Introducción a Matplotlib III

Hay esencialmente dos maneras de graficar con Matplotlib:

- **Estilo pyplot:** simple y rápida para figuras que no son muy avanzadas. Quizás más fácil para empezar.
- **Estilo orientado-objetos:** un poco más complejo pero necesario para figuras que requieren mucha customización.

En cuanto al resultado estético, con ambos se puede lograr la misma calidad. Para dar los primeros pasos es indistinto cual se use. Sin embargo, el estilo orientado a objetos es necesario para figuras más complejas dónde hay varios gráficos (subplots) y es necesario definir parámetros distintos para cada par de ejes (2D).

Preparar el ejemplo I

Poniendo en práctica lo que estuvimos aprendiendo, preparemos una nueva base:

```
1 >>> import pandas as pd
2 >>> # abrimos archivo de potencia energética instalada en
    el país
3 >>> df = pd.read_excel("potencia_instalada.xlsx")
4 >>> # exploren la base. En la segunda hoja esta la fuente.
5 >>> # agregamos a nivel de tipo de fuente
6 >>> df_fuente = df.groupby(by=["periodo", "fuente_generacion "
    ]).agg({"potencia_instalada_mw": "sum"})
7 >>> df_fuente.reset_index(inplace=True)
8 >>> # exploren df_fuente
```

Preparar el ejemplo II

Vamos a graficar dos líneas, así que definimos vector X e Y para cada una. Vamos a graficar la potencia instalada de generación por fuente Renovable y fuente Térmica:

```
1 >>> # defino vectores de datos para serie 1
2 >>> y1 = df_fuente[df_fuente["fuente_generacion"]=="
    Renovable"]["potencia_instalada_mw"]
3 >>> x1 = df_fuente[df_fuente["fuente_generacion"]=="
    Renovable"]["periodo"]
4 >>> # defino vectores de datos para serie 2
5 >>> y2 = df_fuente[df_fuente["fuente_generacion"]=="Termica"
    ]["potencia_instalada_mw"]
6 >>> x2 = df_fuente[df_fuente["fuente_generacion"]=="Termica"
    ]["periodo"]
```

Nota: `df[condicion][columna]` selecciona la columna de la base que resulta de aplicar el filtro `df[condicion]`.

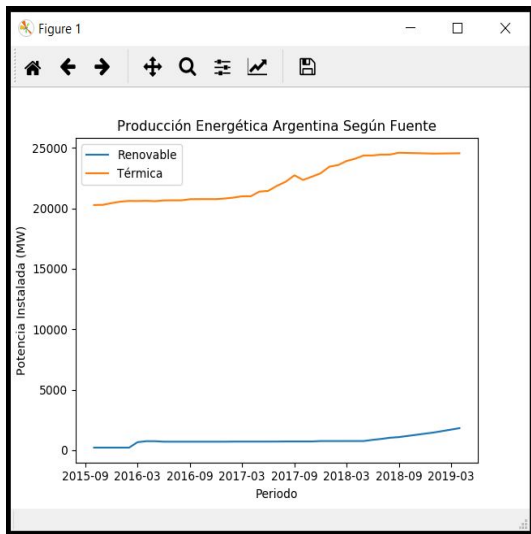
Graficar a la pyplot

Para graficar al estilo pyplot:

```
1 >>> import matplotlib.pyplot as plt
2 >>> # creo el grafico
3 >>> plt.plot(x1,y1, label='Renovable') # serie 1
4 >>> plt.plot(x2, y2, label='Termica') # serie 2
5 >>> #estas dos lineas estaran sobre el mismo grafico
6 >>> # modifico labels
7 >>> plt.xlabel("Periodo")
8 >>> plt.ylabel("Potencia Instalada (MW)")
9 >>> plt.title("Produccion Energetica Argentina Segun Fuente"
10 >>> # agrego leyenda
11 >>> plt.legend()
12 >>> plt.show() #esto es necesario para visualizar
```

Nota: luego de correr `plt.show()` se abre una ventana con el gráfico. Es necesario cerrarla para seguir trabajando sobre la consola.

Graficar a la pyplot: resultado



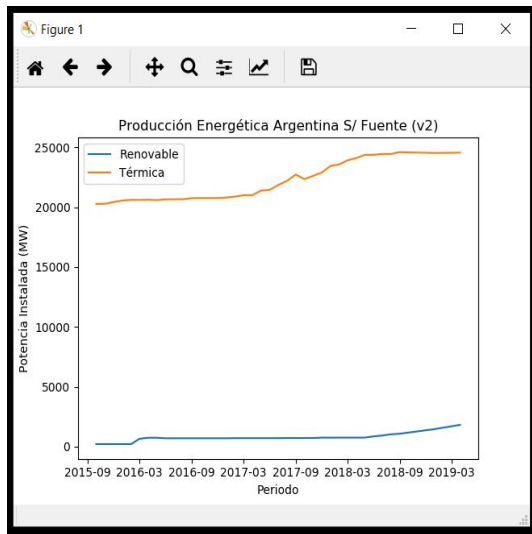
Graficar a la O-O: un gráfico

Para graficar al estilo OO:

```
1 >>> import matplotlib.pyplot as plt
2 >>> # creo el figura y axes
3 >>> fig, ax = plt.subplots() # crear objetos
4 >>> # defino series
5 >>> ax.plot(x1, y1, label="Renovable") # Serie 1
6 >>> ax.plot(x2, y2, label='Termica') # Serie 2
7 >>> # modifico labels
8 >>> ax.set_xlabel('Periodo')
9 >>> ax.set_ylabel('Potencia Instalada (MW)')
10 >>> ax.set_title("Produccion Energetica Argentina S/ Fuente
    (v2)")
11 >>> # agrego leyenda
12 >>> ax.legend()
13 >>> fig.show()
```

Nota: luego de correr `fig.show()` se abre una ventana con el gráfico. Es necesario cerrarla para seguir trabajando sobre la consola.

Graficar a la O-O: un gráfico (resultado)

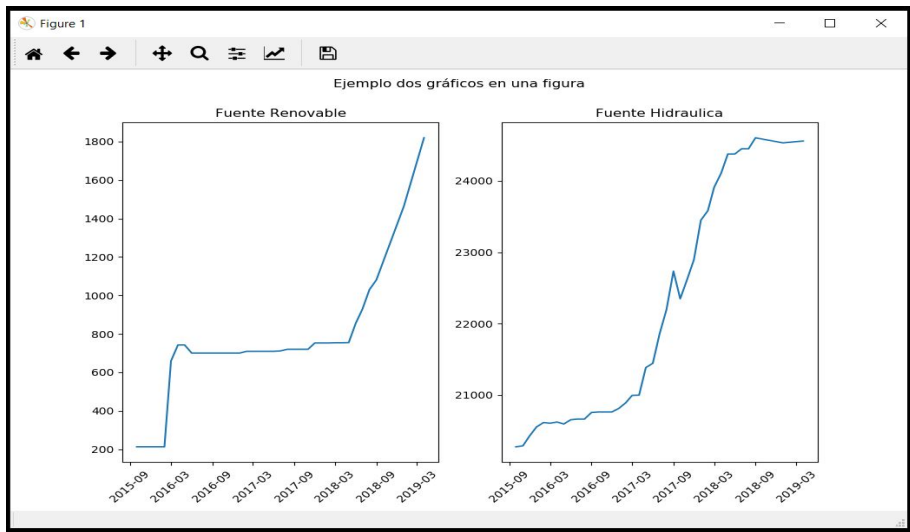


Graficar a la pyplot: múltiples gráficos

Para armar una figura con dos (o más) gráficos:

```
1 >>> import matplotlib.pyplot as plt
2 >>> # ejemplo 2 ax en un fig
3 >>> plt.figure(figsize=(9, 5))
4 >>> # defino primer grafico
5 >>> plt.subplot(121)
6 >>> plt.plot(x1, y1)
7 >>> plt.title('Fuente Renovable')
8 >>> plt.xticks(rotation=45) # valores eje x en diagonal
9 >>> # defino segundo grafico
10 >>> plt.subplot(122)
11 >>> plt.plot(x2, y2)
12 >>> plt.title('Fuente Hidraulica')
13 >>> plt.xticks(rotation=45) # valores eje x en diagonal
14 >>> # definto titulo general
15 >>> plt.suptitle("Ejemplo dos gr ficos en una figura")
16 >>> plt.show()
```

Graficar a la pyplot: dos gráficos (resultados)



Python para Economistas

Universidad Nacional de La Plata

14 de octubre de 2022

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

En la clase pasada vimos:

- ① NumPy
 - Creación de arrays y operaciones.
 - Números aleatorios.
- ② Pandas: manipulación de datos tabulares.
- ③ Matplotlib: generación de gráficos.

Objetivos de la clase

En la clase de esta semana cubriremos los siguientes temas:

- **Profundización en pandas:** vamos a avanzar sobre lo visto en la clase 3 y cubrir algunas operaciones más complejas.
- **Visualización con seaborn:** vamos a estudiar este paquete derivado de `matplotlib`, que tiene una interfaz más amigable para trabajar con `pandas`.
- **Expresiones regulares:** es una herramienta muy poderosa para el procesamiento de texto.
- ***Widgets*:** es una herramienta muy útil para dotar a nuestros notebooks de Jupyter de reactividad.

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

Pandas (continuación de la clase 3)

En la clase 3 vimos:

- Cómo crear un `DataFrame` desde cero.
- Cómo cargar datos de un archivo tabular en un `DataFrame`.
- Cómo filtrar filas y/o columnas.
- Combinar `DataFrame`'s en base a los valores de una o más columnas.
- Agrupar de acuerdo a una o varias columnas y generar agregaciones a nivel de los grupos generados (por ejemplo: cuenta, suma, promedio)

En esta parte de la clase vamos a cerrar el tema de Pandas, viendo algunos conceptos un poco más avanzados.

Otras operaciones con DataFrame's agrupados

En la clase 3, vimos cómo computar estadísticas agregadas a nivel de grupos, utilizando el método `groupby` seguido de una operación de agregación en el objeto resultante.

Otras cosas que podemos querer hacer a nivel de grupos son:

- **Transformaciones.** Por ejemplo:
 - estandarizar una variable de acuerdo a la distribución que tiene condicionada al grupo.
 - rellenar datos faltantes con, por ejemplo, la media del grupo.
- **Filtrado:** descartar observaciones que pertenecen a grupos con determinadas características. Por ejemplo:
 - descartar filas pertenecientes a grupos con pocas observaciones.
 - descartar filas donde un estadístico a nivel de grupo (como el promedio) cumple cierta condición.

Transformaciones sobre grupos

Ejemplo. Estandarizar columnas usando la media y la desviación estándar de cada grupo.

```
1 df.groupby().transform(lambda x: (x - x.mean())/x.std())
```

Ejemplo desarrollado en el notebook.

Filtrado por grupos

Ejemplo. Eliminar filas pertenecientes a grupos con menos de 5 observaciones:

```
1 df.groupby().filter(lambda x: len(x) < 5)
```

Ejemplo desarrollado en el notebook.

Pandas: reformateando DataFrame's

La forma canónica de estructurar un **DataFrame** es poner las *observaciones* en las filas, y las distintas *variables* observadas en las columnas.

Sin embargo, qué se considera una observación exactamente no está bien definido, y la estructura que le demos a nuestro **DataFrame** va a quedar determinada en gran medida por lo que queramos hacer después.

Pandas: reformateando DataFrame's

Pasar de un DataFrame 'ancho' a uno 'largo':

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



df3.melt(id_vars=['first', 'last'])

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

Pregunta 1 (Cuestionario)

Supongamos que tenemos el DataFrame `df`:

	Nombre	Apellido	Edad	Dirección	Ciudad
0	Mabel	García	40	San Martín 900	La Plata
1	Juan	Pérez	50	Belgrano 1770	Rosario
2	Héctor	Rossi	60	Sarmiento 1888	Santa Rosa

¿Qué dimensiones (`shape`) tiene el resultado de hacer `df.melt(id_vars=['Nombre', 'Apellido'])`? Expresarlo como `filas × columnas`.

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

Visualizaciones (continuación)

Vamos a estudiar el paquete **seaborn** para gráficos.

seaborn está basado en **matplotlib** y ofrece una interfaz más amigable para trabajar con objetos como **DataFrame**'s de **pandas**. Esto permite realizar gráficos relativamente complejos con un código relativamente simple.

Lo podemos instalar de esta manera:

```
conda install -c anaconda seaborn
```

Una vez instalado, lo importamos así¹:

```
1 import seaborn as sns
```

¹Como siempre, usar el alias al importar no es necesario pero es una práctica habitual.

Un gráfico sencillo con seaborn

```
1 sns.set_theme()
```

Pregunta 2 (cuestionario)

Dado el `DataFrame` de propinas (supongamos que se llama `propinas_df`) que vimos en el ejemplo:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

¿Qué línea de código en **seaborn** escribirías para generar un boxplot que tenga en el eje horizontal el tamaño de la mesa y en el eje vertical el valor de la propina?

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares**
- 4 Widgets en Jupyter

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)
- Validación de entrada de usuario (por ejemplo, verificar que un string es una dirección de e-mail válida).

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)
- Validación de entrada de usuario (por ejemplo, verificar que un string es una dirección de e-mail válida).
- Minado de datos.

Expresiones regulares

Una *expresión regular* es una secuencia de caracteres que especifica un patrón de texto.

Son ampliamente usadas, por ejemplo, en:

- Procesadores de texto (*buscar* o *buscar y reemplazar*).
- Compiladores o intérpretes de lenguajes de programación (por ejemplo, el intérprete de Python!)
- Validación de entrada de usuario (por ejemplo, verificar que un string es una dirección de e-mail válida).
- Minado de datos.

Expresiones regulares

En Python, el módulo `re` se utiliza para trabajar con expresiones regulares. No hace falta instalarlo, pues viene incluido en la librería estándar.

Para trabajar con él, ejecutamos

```
1 import re
```

¿Cómo defino una expresión regular?

Empecemos con una expresión regular sencilla:

```
1 >>> import re
2 >>> frase = "    Dnde    va Vicente? Donde va la gente."
3 >>> regex = re.compile("ente")
4 >>> regex.search(frase)
5 <re.Match object; span=(13, 17), match='ente'>
```

Sigue en el notebook...

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, [`<caracteres>`], donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, [`<caracteres>`], donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Si se quiere expresar que un carácter **no** debe pertenecer a un conjunto, se utiliza "[`^<caracteres>`]".

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, `[<caracteres>]`, donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Si se quiere expresar que un carácter **no** debe pertenecer a un conjunto, se utiliza `"[<caracteres>]"`.

Por ejemplo:

- la expresión regular `"[Hhbc]ola"` matchea con `Hola`, `hola`, `bola`, `cola` y `sola`.

Expresiones regulares: conjuntos de caracteres

Para especificar conjuntos de caracteres dentro de una expresión regular, se utilizan los corchetes y dentro de ellos se ponen todas las opciones de caracteres, `[<caracteres>]`, donde `caracteres` son todos los caracteres del conjunto, uno al lado del otro.

Si se quiere expresar que un carácter **no** debe pertenecer a un conjunto, se utiliza `[^<caracteres>]`.

Por ejemplo:

- la expresión regular `"[Hhbc]ola"` matchea con `Hola`, `hola`, `bola`, `cola` y `sola`.
- la expresión regular `"[^bc]ola"` matchea con `hola` y `sola`, pero no con `bola` y `cola`.

Expresiones regulares: conjuntos de caracteres

Algunos conjuntos de caracteres se pueden expresar de manera abreviada dentro de los corchetes, por ejemplo:

[a-z]: una letra minúscula

[A-Z]: una letra mayúscula

[a-zA-Z]: una letra

[0-9]: dígito

Expresiones regulares: conjuntos de caracteres

Algunos conjuntos de caracteres se pueden expresar de manera abreviada dentro de los corchetes, por ejemplo:

`[a-z]`: una letra minúscula

`[A-Z]`: una letra mayúscula

`[a-zA-Z]`: una letra

`[0-9]`: dígito

También, existen conjuntos de caracteres predefinidos que se pueden expresar sin necesidad de corchetes, por ejemplo:

`\s`: un carácter de espacio, equivalente a `[\t\n\r\f\v]`

`\d`: un dígito

`\w`: un carácter de palabra

`\W`: la negación de `\w`

Expresiones regulares: caracteres especiales

Hay un conjunto de caracteres que tienen un significado especial dentro de una expresión regular (es decir, no se representan a sí mismos).

Éstos son:

. * ? + ^ \$ \ | () [] { }

Es conveniente memorizarlos. En lo siguiente vamos a ver el significado de cada uno.

Expresiones regulares: caracteres especiales

Hay un conjunto de caracteres que tienen un significado especial dentro de una expresión regular (es decir, no se representan a sí mismos).

Éstos son:

. * ? + ^ \$ \ | () [] { }

Es conveniente memorizarlos. En lo siguiente vamos a ver el significado de cada uno.

¿Y si realmente queremos usar el significado literal de estos caracteres?
Escribimos "\", antes del carácter especial. Esto se llama *escaping*.

Expresiones regulares: caracteres especiales

- Comienzo y fin de un string: $\wedge$ y $\$$, respectivamente.
- El carácter comodín: $.$
- Cuantificadores: $*$, $+$, $?$.
 - $*$ $\rightarrow$ 0 ó más ocurrencias.
 - $+$ $\rightarrow$ 1 ó más ocurrencias.
 - $?$ $\rightarrow$ 0 ó 1 ocurrencia.

Expresiones regulares: caracteres especiales

- Comienzo y fin de un string: $\wedge$ y $\$$, respectivamente.
- El carácter comodín: $.$
- Cuantificadores: $*$, $+$, $?$.
 - $*$ $\rightarrow$ 0 ó más ocurrencias.
 - $+$ $\rightarrow$ 1 ó más ocurrencias.
 - $?$ $\rightarrow$ 0 ó 1 ocurrencia.

Por ejemplo, la expresión regular `".*"` matchea con cualquier string.

Expresiones regulares: caracteres especiales

- El `|` separando dos expresiones regulares representa un **or** lógico.

Métodos de expresiones regulares

Ahora que vimos cómo definir expresiones regulares, veamos los métodos asociados. En lo sucesivo,

- `patron` es un string que representa una expresión regular,
- `regex` es un objeto de la clase `re.Pattern` y
- `texto` es un string donde buscamos el patrón `patron`.

Métodos de expresiones regulares

- `re.compile(patron)`: me genera y devuelve el objeto de tipo `re.Pattern` asociado a `patron`.
- `re.findall(patron, texto)` o `regex.findall(texto)`: devuelve una lista con todas las ocurrencia de `regex` en `texto`.
- `re.match(patron, texto)` o `regex.match(texto)`: devuelve un objeto de tipo `re.Match` si `patron/regex` *matchea* con `texto`, o `None` en caso contrario².
- `re.finditer(patron, texto)` o `regex.finditer(texto)`: devuelve un iterador (i.e. un objeto sobre el cual podemos iterar por ejemplo con un `for`), cada uno de cuyos elementos es un objeto `re.Match` correspondiente a cada una de las ocurrencias de `patron/regex`.

²Notar que el texto **completo** debe matchear con `patron/regex`, no es suficiente que el patrón esté simplemente contenido en `texto`

Métodos de expresiones regulares

- `re.split(patron, texto)` o `regex.split(texto)`: devuelve una lista de `string`'s que resultan de particionar `texto` por todas las ocurrencias de `patron/regex`.
- `re.sub(repl, patron, texto)` o `regex.sub(repl, texto)`: devuelve el `string` que resulta de sustituir todas las ocurrencias de `patron/regex` por el `string repl`.

Extrayendo *grupos* de un string

Dada una expresión regular compuesta por distintas sub-expresiones regulares encerradas entre paréntesis, se pueden extraer las distintas porciones del string que satisfagan cada una:

```
1 >>> import re
2 >>> regex = "(.*)\.(txt|csv|xls|doc)"
3 >>> regex = re.compile(regex)
4 >>> prueba = "archivo_imaginario.xls"
5 >>> nombre = regex.match(prueba).group(1)
6 >>> nombre
7 "archivo_imaginario"
8 >>> extension = regex.match(prueba).group(2)
9 >>> extension
10 "xls"
```

Pregunta 3 (cuestionario)

- a. Escribir una expresión regular que *matchee* con un string que *contiene* una letra **a** seguida de cualesquiera caracteres, y termina en una letra **b**.

- b. Escriba una expresión regular que *matchee* con un string que *contiene* una o más letras **a** seguidas inmediatamente de cero o más dígitos.

Temario de la clase

- 1 Pandas (continuación de la clase 3)
- 2 Visualizaciones (continuación de la clase 3)
- 3 Expresiones regulares
- 4 Widgets en Jupyter

Widgets

Los widgets (o *artilugios*) son objetos interactivos que permiten especificar determinados valores utilizando una interfaz gráfica amigable.

Es posible utilizar widgets en Jupyter, a través del módulo `ipywidgets`, el cual se puede instalar desde la consola con `pip` o `conda`:

```
conda install -c anaconda ipywidgets
```

Para utilizar los widgets, se debe importar el módulo `iwidgets` de esta manera:

```
1 import ipywidgets as widgets
```

Notar que el código de esta sección se debe ejecutar en Jupyter notebooks, no en la consola.

Widgets

```
1 import ipywidgets as widgets
2 w = widgets.IntSlider()
3 display(w)
```



Para obtener el valor asociado al widget, se puede hacer

```
1 w.value
2 4
```

Widgets

Podemos dotar a nuestros widgets de reactividad mediante la función `interact`:

```
1 import ipywidgets as widgets
2 from ipywidgets import interact
3
4 def f(x):
5
6     #####
7     # Lo que queremos hacer
8     #####
9
10 interact(f, x=widgets.IntSlider(min=0, max=10))
```

¡Fin de la clase 4!

Curso de Python para Economistas

Universidad Nacional de La Plata

Octubre, 2022

Temario de la clase

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Repaso de la última clase

En la clase 4 vimos:

- Continuamos trabajando con tablas de datos estructurados (clase 3)
- Profundización en Pandas
- Visualizaciones con seaborn
- Utilización de widgets
- Expresiones Regulares

Objetivos de la clase

- La clase de esta semana presenta distintas herramientas útiles para acceder a diversas fuentes de datos.
- Se busca exponer los paquetes más utilizados para hacer web scraping, pdf scraping, conexión a APIs, etc.
- Además, se conversará acerca de las dificultades que pueden aparecer en cada proyecto en particular y que estrategias pueden servir para sortearlas.
- Se espera que estas herramientas sean útiles para quienes esten pensando en sus proyectos de investigación/tesis.

Liberías principales

Algunas de las librerías más utilizadas para los propósitos de esta clase son:

- **Requests:** Permite hacer solicitudes de HTTP
- **BeautifulSoup4:** permite extraer información de paginas web
- **PyPDF2:** simplifica la manipulación de PDFs
- **Tabula:** extrae tablas de PDFs
- **Minecart:** facilita la extraccion de texto, imagenes, colores y formas de PDFs.
- **Sqlite3:** permite conectarse a bases de datos relacionales

Para poder seguir esta clase a la par, con la consola o un Jupyter Notebook, por favor instalen las librerías indicadas en el documento que acompaña esta presentación.

- PDF Scraping
- Web Scraping
- Descargar con un click (csv, excel)
- Conectarnos a una API
- Base de datos relacionales (tipo SQL)

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Descargar con un click

Con el paquete 'requests' podemos enviar solicitudes HTTP/1.1. para obtener el contenido de una url. Es como si estuviéramos accediendo a la url desde el navegador y descargando el contenido del archivo.

```
1 >>> def descarga_base_infectados(csv_url, csv_name):
2     #Obtenemos el contenido del archivo en el url
3     #especificado
4     req = requests.get(csv_url)
5     url_content = req.content
6
7     #Guardamos el contenido descargado en un archivo
8     #con el nombre que elegimos en 'csv_name'
9     csv_file = open(csv_name, 'w')
10    csv_file.write(url_content.decode())
11    csv_file.close()
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Web scraping: ¿Qué es?

- 'Web scraping' es la extracción de datos de sitios web.
- Es una herramienta para obtener información de forma programática y reproducible
- Utilizaremos web scraping cuando queramos datos de un sitio web que no tenga API
- Hay que ser cuidadosos que nuestro web scraping no este violando ninguna regla de privacidad o del sitio
- La calidad de los datos que extraigamos va a depender de la calidad del sitio
- El navegador utiliza instrucciones en HTML para mostrar el sitio web.
- Detectamos el patrón en el HTML y definimos las secciones a extraer

Web scraping: ¿Qué es un HTML?

- HTML son las siglas de Hyper Text Markup Language
- HTML es el lenguaje de marcado estándar para crear páginas web
- HTML describe la estructura de una página web
- HTML consta de una serie de elementos que le dicen al navegador cómo mostrar el contenido.
- Los elementos HTML etiquetan partes de contenido para indicar "esto es un encabezado", "esto es un párrafo", "esto es un enlace", etc.

Web scraping

<code><!DOCTYPE></code>	Sirve para definir el tipo de documento
<code><html></html></code>	Define un documento HTML
<code><head></head></code>	Dentro se escribe información sobre el documento
<code><title></title></code>	Para escribir el título del documento. Se escriben dentro de <code><head></code>
<code><body></body></code>	Justo después de <code></head></code> . Es el contenedor del documento
<code><h1></h1></code> , <code><h2></h2></code> hasta <code><h6></h6></code>	Para crear títulos de párrafos, de más importante <code><h1></code> a menos <code><h6></code>
<code><p></p></code>	Los párrafos de texto de nuestro documento
<code>
</code>	Un salto de línea simple
<code><hr></code>	Salto de línea que indica un cambio de temática entre párrafos
<code><a></code>	Podremos escribir enlaces a otras páginas

fuelle: <https://websgalicia.es/blog/etiquetas-html/>

Pueden consultar otras etiquetas (tags) en **CheetSheets de HTML**

Web scraping

DataTables Editor CloudTables

[Manual](#) [Download](#) [Login / Register](#)

Using DataTables

Examples

Basic initialisation

[Advanced initialisation](#)

[Styling](#)

[Data sources](#)

[API](#)

[Ajax](#)

[Server-side](#)

[Plug-ins](#)

Manual

Reference

Extensions

Plug-ins

[More Help](#)

[Blog](#)

[Forums](#)

[Support](#)

[FAQs](#)

[Get DataTables](#)

[Download](#)

[Purchase](#)

Zero configuration

DataTables has most features enabled by default, so all you need to do to use it with your own tables is to call the construction function: `$(.`

Searching, ordering and paging goodness will be immediately added to the table, as shown in this example.

Show entries

Search:

Name	Position	Office	Age	Start date	Salary
Airi Satou	Accountant	Tokyo	33	2008/11/28	\$162,700
Angelica Ramos	Chief Executive Officer (CEO)	London	47	2009/10/09	\$1,200,000
Ashton Cox	Junior Technical Author	San Francisco	66	2009/01/12	\$86,000
Bradley Greer	Software Engineer	London	41	2012/10/13	\$132,000
Brenden Wagner	Software Engineer	San Francisco	28	2011/06/07	\$206,850
Brielle Williamson	Integration Specialist	New York	61	2012/12/02	\$372,000
Bruno Nash	Software Engineer	London	38	2011/05/03	\$163,500
Caesar Vance	Pre-Sales Support	New York	21	2011/12/12	\$106,450
Cara Stevens	Sales Assistant	New York	46	2011/12/06	\$145,600
Cedric Kelly	Senior Javascript Developer	Edinburgh	22	2012/03/29	\$433,060

Name	Position	Office	Age	Start date	Salary
Showing 1 to 10 of 57 entries					
			Previous	1	2 3 4 5 6 Next

Sitio: https://datatables.net/examples/basic_init/zero_configuration.html

Web scraping

```
<!DOCTYPE html>
<html>
  <head>...</head>
  <body class="wide comments example" data-new-gr-c-s-check-loaded="14.983.0">
    <a name="top" id="top">...</a>
    <div class="fw-background">...</div>
    <div class="fw-container">
      <div class="fw-header">...</div>
      <div class="fw-nav">...</div>
      <div class="fw-body">
        <div class="content">
          <h1 class="page_title">Zero configuration</h1>
          <div class="info" style="min-height: 8em;">...</div>
          <div class="demo-html">
            <div id="example_wrapper" class="dataTables_wrapper">
              <div class="dataTables_length" id="example_length">...</div>
              <div id="example_filter" class="dataTables_filter">...</div>
              <table id="example" class="display dataTable" style="width: 100%;" role="grid" aria-describedby="example_info">
                <thead>...</thead>
                <tbody>
                  <tr role="row" class="odd">
                    <td class="sorting_1">Airi Satou</td>
                    <td>Accountant</td>
                    <td>Tokyo</td>
                    <td>33</td>
                    <td>2008/11/28</td>
                    <td>$162,700</td>
                  </tr>
                  <tr role="row" class="even">
                    <td class="sorting_1">Angelica Ramos</td>
                    <td>Chief Executive Officer (CEO)</td>
                    <td>London</td>
                    <td>47</td>
                    <td>2009/10/09</td>
                    <td>$1,200,000</td>
                  </tr>
                  <tr role="row" class="odd">
                    <td class="sorting_1">Ashton Cox</td>
                    <td>Junior Technical Author</td>
                    <td>San Francisco</td>
                    <td>66</td>
                    <td>2009/01/12</td>
                    <td>$86,000</td>
                  </tr>
                </tbody>
              </table>
            </div>
          </div>
        </div>
      </div>
    </body>
  </html>
```

Web scraping

```
1 >>> import requests
2 >>> from bs4 import BeautifulSoup
3 >>> import pandas as pd
```

```
1 >>> url = 'https://datatables.net/examples/basic_init/
    zero_configuration.html'
```

```
1 >>> response = requests.get(url)
2 >>> soup = BeautifulSoup(response.text, 'html.parser')
3 >>> info = soup.find_all('tr')
```

Web scraping

```
1 >>> print(info)
2 [<tr>
3 <th>Name</th>
4 <th>Position</th>
5 <th>Office</th>
6 <th>Age</th>
7 <th>Start date</th>
8 <th>Salary</th>
9 </tr>, <tr>
10 <td>Tiger Nixon</td>
11 <td>System Architect</td>
12 <td>Edinburgh</td>
13 <td>61</td>
14 <td>2011/04/25</td>
15 <td>$320,800</td>
16 </tr>, <tr>
17 <td>Cedric Kelly</td>
```

Web scraping

```
1 >>> df = pd.DataFrame(columns=['name', 'position', 'office',  
2                               'age', 'start', 'salary'])
```

```
1 >>> for i, item in enumerate(info):  
2     ...     if i != 0 and i != len(info)-1:  
3     ...         data_point = item.find_all('td')  
4     ...         row = []  
5     ...         for value in data_point:  
6     ...             row.append(value.text)  
7     ...         df.loc[i-1] = row
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping**
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Utilizaremos el siguiente PDF: **Boletín IFE - ANSES**

Utilizamos el paquete PyPDF2

```
1 >>> import PyPDF2
```

Utilizamos el metodo 'PdfFileReader' y vamos imprimir el contenido leído en cada pagina con un loop y los metodos 'getPage' y 'extractText'

```
1 >>> pdf_file = open('boletin_ife.pdf', 'rb')
2 >>> pdfReader = PyPDF2.PdfFileReader(pdf_file)
3 >>> paginas = pdfReader.numPages
4 >>> for i in range(paginas):
5 ...     print("Contenido de la pagina {}".format(i))
6 ...     pagina = pdfReader.getPage(i)
7 ...     print(pagina.extractText())
```

Para extraer las tablas del PDF vamos a usar el paquete tabula

```
1 >>> import tabula
2 >>> import pandas as pd
```

Utilizamos el metodo 'read_pdf', le vamos a solicitar la tabla de la pagina 7 y que la devuelva como dataframe

```
1 >>> lst_of_df = tabula.read_pdf('boletin_ife.pdf',
    output_format='dataframe', pages=7)
```

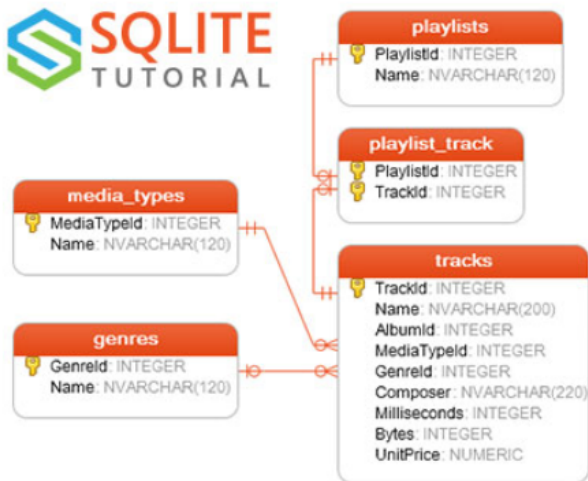
```
1 >>> tabla_p7 = lst_of_df[0]
2 >>> tabla_p7.columns = ['tipo_de_hogar', 'pobreza', '
    indigencia']
```

Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)**
- 6 Conexión a una API

Base de Datos relacionales

Figure: Base de datos relacional



fuelle: <https://www.sqlitetutorial.net/>

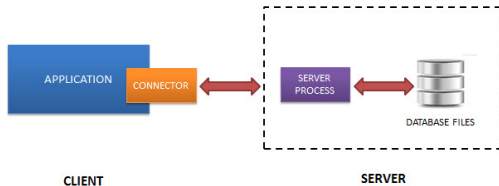
Base de Datos relacionales

Figure: SQLite: No requiere servidor



Self-Contained

Figure: PostgreSQL: Requiere un servidor de BD



fuentes: [sqlite](#) vs [postgresql](#)

Figure: SQLite: Comandos de selección de datos

<code>SELECT *</code>	<i>#Que se quiere seleccionar</i>
<code>FROM hogares_uni</code>	<i>#De que tabla</i>
<code>WHERE total > 10000</code>	<i>#Condición</i>
<code>ORDER BY provincia;</code>	<i>#Variable que ordena</i>

Figure: SQLite: Comandos de edición de tablas

<code>CREATE</code>	<i>#Para crear una tabla</i>
<code>DROP</code>	<i>#Para eliminar una tabla</i>
<code>INSERT</code>	<i>#Para agregar obs. a la tabla</i>

Base de Datos relacionales

Utilizamos el paquete sqlite3

```
1 >>> import sqlite3
```

Iniciamos la conexión a la base de datos con el metodo 'connect' y creamos un objeto cursor para poder comenzar a ejecutar los comandos de SQL:

```
1 >>> conn = sqlite3.connect('IFE.db')  
2 >>> c = conn.cursor()
```

Creo la tabla, inserto una observacion a la tabla

```
1 >>> c.execute('''CREATE TABLE provincias (provincia text,  
2               prov_cod INTEGER PRIMARY KEY)''')  
3 >>> c.execute("INSERT INTO provincias VALUES ('Buenos Aires  
    ', 1)")
```

Base de Datos relacionales

Insertamos observaciones en bloque a la tabla

```
1 >>> provincias = [('CABA', 2),  
2                   ('Catamarca', 3),  
3                   ('Chaco', 4)]  
4 >>> c.executemany('INSERT INTO provincias VALUES (?,?)',  
                   provincias)
```

Guardamos (commit) los cambios y cerramos la conexión

```
1 >>> conn.commit()  
2 >>> conn.close()
```


Table of Contents

- 1 Repaso clase 4 y objetivos clase 5
- 2 Descargar con un click
- 3 Web scraping
- 4 PDF scraping
- 5 Base de datos relacionales (tipo SQL)
- 6 Conexión a una API

Conexión a una API

- Las API (application programming interface) son una interfaz informática que define las interacciones entre múltiples intermediarios de software.
- Define los tipos de llamadas o solicitudes que se pueden realizar, cómo hacerlas, los formatos de datos que deben usarse, las convenciones a seguir, etc.
- También puede proporcionar mecanismos de extensión para que los usuarios puedan extender la funcionalidad existente de varias formas y en diferentes grados.
- Las API muchas veces requieren autenticación para garantizar la seguridad de los datos.

Gracias

Esperamos sus dudas en Slack

Curso de Python para Economistas

Universidad Nacional de La Plata

23 Octubre, 2022

Temario de la clase

- 1 Repaso de la última clase
- 2 Conceptos básicos de georreferenciación
- 3 Introducción a Geopandas
- 4 Elaboración de mapas

Contenido de esta clase

- 1 Repaso de la última clase
- 2 Conceptos básicos de georreferenciación
- 3 Introducción a Geopandas
- 4 Elaboración de mapas

Repaso de la última clase

En la clase 5 vimos:

- Peticiones http (módulo `requests`)
- Scraping de páginas web o PDFs
 - BeautifulSoup
 - PyPDF2
 - Tabula
- Bases de datos relacionales
- Conexión a un servidor a través de una API.

Objetivos de la clase

- Familiarizarnos con los conceptos básicos sobre georreferenciación.
- Aprender a usar la librería Geopandas.
- Aprender a crear mapas estáticos e interactivos.

Contenido de esta clase

- 1 Repaso de la última clase
- 2 Conceptos básicos de georreferenciación
- 3 Introducción a Geopandas
- 4 Elaboración de mapas

Georreferenciación

En esta clase vamos a trabajar con información georreferenciada: objetos, cuyos atributos están vinculados a una capa espacial a partir de coordenadas.

Sistema de información geográfica (SIG)

El SIG (o GIS en inglés) es un marco conceptual que establece cómo capturar, almacenar, analizar y representar información espacial y geográfica.

Hay dos tipos de datos principales:

- **Raster:** el objeto de interés se representa a través de una grilla. Usado para representar fenómenos continuos a través del color de los pixeles de una imagen (al estilo de un mapa de calor o *heatmap*); por ejemplo, altitud o temperatura.
- **Vectorial:** el objeto de interés se representa a través de formas geométricas. Es en el que nos vamos a enfocar aquí.

Para modelar digitalmente las entidades del mundo real se utilizan tres elementos geométricos: el punto, la línea y el polígono (y sus versiones compuestas: multipunto, multilínea y multipolígono).

- **Punto:** para representar objetos donde la extensión no es importante, sólo la ubicación. No tienen ningún atributo espacial asociado (aparte de la ubicación misma).
- **Línea:** para representar objetos “unidimensionales” como ríos, calles, rutas, etc. Tienen una *distancia* asociada.
- **Polígonos:** para representar objetos “bidimensionales”, como un país, una provincia u otra jurisdicción. Tienen asociados un *perímetro* y un *área*.

Formatos de datos georreferenciados vectoriales

Algunos de los formatos de archivo que se usan son:

- **shp** (*shape files*): formato propietario desarrollado por Esri (aunque existen implementaciones de código abierto). Típicamente está acompañado de otros 2 ó 3 tipos de archivo que sirven como índices/metadatos: `.shx`, `.dbf` y `.sbn`. Pueden ver [aquí](#) las especificaciones.
- **GeoJSON**: es un estándar abierto basado en JSON. Ver por ejemplo [esta página](#) para crear archivos GeoJSON propios desde cero.
- **KML** (Keyhole Markup Language): basado en el formato XML. Desarrollado por Google Earth.

Hay distintas maneras de representar gráficamente porciones de la superficie terrestre como porciones de planos. Esto se conoce como **proyección cartográfica**.

Hay distintas maneras de representar gráficamente porciones de la superficie terrestre como porciones de planos. Esto se conoce como **proyección cartográfica**.

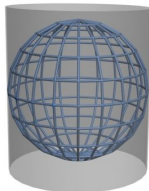
Esto va a ser tanto más relevante cuanto mayor sea el área que se quiere representar (es decir, cuanto menos se aproxime la superficie representada a una superficie plana).

Proyecciones

Hay distintas maneras de representar gráficamente porciones de la superficie terrestre como porciones de planos. Esto se conoce como **proyección cartográfica**.

Esto va a ser tanto más relevante cuanto mayor sea el área que se quiere representar (es decir, cuanto menos se aproxime la superficie representada a una superficie plana).

Proyección de Mercator: es una de las más utilizadas para representar el globo terráqueo completo, y proyecta la superficie terrestre sobre un cilindro:



Estándares de referencia

WGS84 (World Geodetic System 84): es el más utilizado, es la base sobre la que se representa la información de GPS.

POSGAR 07: o POSGAR 94 en su versión anterior, es el sistema de coordenadas de referencia oficial de Argentina.

Sistemas de coordenadas de referencia

Al cargar información georreferenciada es importante revisar los metadatos de la base para verificar el sistema de coordenadas (CRS por sus siglas en inglés) utilizado.

El sistema de coordenadas puede ser:

- **Cartesiano:** mapeando valores para X e Y.
- **Esférico:** usando latitud y longitud.

Los sistemas de coordenadas definen también la unidad de medida de distancias y áreas.

Fuentes de datos georreferenciados

En Argentina:

- Instituto Geográfico Nacional (IGN): ver [esta página](#).

Contenido de esta clase

- 1 Repaso de la última clase
- 2 Conceptos básicos de georreferenciación
- 3 Introducción a Geopandas**
- 4 Elaboración de mapas

Georreferenciación en Python

Existen programas pagos (ej. ArcGIS) y gratuitos (ej. QGIS) para trabajar con información georreferenciada. Estas son herramientas con funcionalidades avanzadas y en algunos casos pueden ser más eficientes para el armado de mapas individuales con bases relativamente pequeñas.

Entonces, ¿por qué es útil aprender a mapear con Python? Porque con Python:

- tenemos la posibilidad de trabajar sobre un mayor volumen de datos sin sacrificar capacidad analítica,
- podemos automatizar la creación de mapas estandarizados para múltiples unidades espaciales,
- podemos desarrollar mapas interactivos.

Geopandas

GeoPandas es un paquete que extiende las funcionalidades de **Pandas** para trabajar con datos georreferenciados.

Para instalar este paquete, correr:

```
conda install -c conda-forge geopandas
```

Para trabajar con él, hacemos:

```
import geopandas
```

o bien

```
import geopandas as gpd
```

Geopandas: tipos de datos

Los dos tipos de objetos que introduce Geopandas son:

- GeoSeries
- GeoDataFrame

Geopandas: tipos de datos

Una **GeoSeries** es esencialmente un vector donde cada elemento es un conjunto de "shapes" (una o varias) que corresponden a una única observación.

Geopandas: tipos de datos

Un `GeoDataFrame` es básicamente un `DataFrame`, que contiene una `GeoSeries` (puede contener más de una también).

En realidad es más que un `DataFrame`, ya que contiene atributos y métodos adicionales, pero la estructura subyacente es la del `DataFrame`.

Geopandas: tipos de datos

Un `GeoDataFrame` es básicamente un `DataFrame`, que contiene una `GeoSeries` (puede contener más de una también).

En realidad es más que un `DataFrame`, ya que contiene atributos y métodos adicionales, pero la estructura subyacente es la del `DataFrame`. En los términos de POO, un `GeoDataFrame` es una *subclase* de la clase `DataFrame`.

Geopandas: tipos de datos

Un **GeoDataFrame** es básicamente un **DataFrame**, que contiene una **GeoSeries** (puede contener más de una también).

En realidad es más que un **DataFrame**, ya que contiene atributos y métodos adicionales, pero la estructura subyacente es la del **DataFrame**. En los términos de POO, un **GeoDataFrame** es una *subclase* de la clase **DataFrame**.



Tipos de datos geométricos

Los objetos geográficos que utiliza Geopandas son de 3 clases, los cuales en realidad hereda del paquete `shapely`:

- `Points` / `Multi-Points`
- `Lines` / `Multi-Lines`
- `Polygons` / `Multi-Polygons`

Pregunta 1 (cuestionario)

¿Qué código escribirías para leer el archivo shapefile (shp) de ciudades del mundo y quedarte con aquellas que pertenecen a Argentina?

Recordar que el nombre correspondiente al país en el GeoDataFrame del mundo (`naturalearth_lowres`) es `"name"`.

Contenido de esta clase

- 1 Repaso de la última clase
- 2 Conceptos básicos de georreferenciación
- 3 Introducción a Geopandas
- 4 Elaboración de mapas

Elaboración de mapas

Geopandas provee una interfaz de alto nivel al paquete Matplotlib para generar mapas. Es tan fácil como ejecutar `gdf.plot()`.
(Sigue en el notebook de esta clase).

Pregunta 2 (cuestionario)

El área en km^2 de cada país en el GeoDataFrame `mundo_gdf` se puede calcular como

```
mundo_gdf.to_crs('proj':'cea').area / 10**6
```

Sabiendo esto, ¿qué código escribirías para generar un mapa donde cada país esté pintado de acuerdo a su densidad de población?